Old Dominion University

ODU Digital Commons

Electrical & Computer Engineering Theses & Dissertations

Electrical & Computer Engineering

Spring 2024

Time Series Models for Predicting Application GPU Utilization and Power Draw Based on Trace Data

Dorothy Xiaoshuang Parry
Old Dominion University, dorothyparry3@gmail.com

Follow this and additional works at: https://digitalcommons.odu.edu/ece_etds

Part of the Artificial Intelligence and Robotics Commons, Bioinformatics Commons, Biostatistics Commons, and the Computer Engineering Commons

Recommended Citation

Parry, Dorothy X.. "Time Series Models for Predicting Application GPU Utilization and Power Draw Based on Trace Data" (2024). Master of Science (MS), Thesis, Electrical & Computer Engineering, Old Dominion University, DOI: 10.25777/xccw-y757
https://digitalcommons.odu.edu/ece_etds/262

This Thesis is brought to you for free and open access by the Electrical & Computer Engineering at ODU Digital Commons. It has been accepted for inclusion in Electrical & Computer Engineering Theses & Dissertations by an authorized administrator of ODU Digital Commons. For more information, please contact digitalcommons@odu.edu.

TIME SERIES MODELS FOR PREDICTING APPLICATION GPU UTILIZATION AND POWER DRAW BASED ON TRACE DATA

by

Dorothy Xiaoshuang Parry
B.S. June 2018, Randolph-Macon College

A Thesis Submitted to the Faculty of
Old Dominion University in Partial Fulfillment of the
Requirements for the Degree of

MASTER OF SCIENCE

MODELING AND SIMULATION ENGINEERING

OLD DOMINION UNIVERSITY
May 2024

Approved by:

Masha Sosonkina (Director)

Yaohang Li (Member)

Katie Smith (Member)

ABSTRACT

TIME SERIES MODELS FOR PREDICTING APPLICATION GPU UTILIZATION AND POWER DRAW BASED ON TRACE DATA

Dorothy Xiaoshuang Parry
Old Dominion University, 2024
Director: Dr. Masha Sosonkina

This work explores collecting performance metrics and leveraging various statistical and machine learning time series predictive models on a memory-intensive application, *Inception v3*. Trace data collected using *nvidia-smi* measured GPU utilization and power draw for two runs of Inception3. Experimental results from the statistical and machine learning-based time series predictive algorithms showed that the predictions from statistical-based models were unable to capture the complex changes in the trace data. The Probabilistic TNN model provided the best results for the power draw trace, according to the test evaluation metrics. For the GPU utilization trace, the RNN models produced the most accurate predictions in terms of MSE and MAE.

Copyright, 2024, by Dorothy Xiaoshuang Parry, All Rights Reserved.

I dedicate this thesis to my family and friends who have supported me.

ACKNOWLEDGMENTS

This work was supported in part by the U.S. Department of Energy (DOE) Office of Science, Office of Basic Energy Sciences, Computational Chemical Sciences (CCS) Research Program under work proposal number AL-18-380-057 through the Ames Laboratory, operated by Iowa State University under contract No. DE-AC00-07CH11358, in part by the Old Dominion University Data Science Seed Funding 2022 Award and in part by the Old Dominion University Research and Cloud Computing. This work also was supported in part by a National Training & Simulation Association (NTSA) Annual RADM Fred Lewis Postgraduate MODSIM World Scholarship received in May 2023.

TABLE OF CONTENTS

Page

LIST OF TABLES vii

LIST OF FIGURES viii

Chapter

1. INTRODUCTION 1
   - 1.1 MOTIVATION 1
   - 1.2 OBJECTIVES 2
   - 1.3 RELATED WORKS 2
   - 1.4 THESIS ORGANIZATION 4

2. BACKGROUND 5
   - 2.1 POWER CONSUMPTION 5
   - 2.2 PERFORMANCE MEASURES 5
   - 2.3 TIME SERIES PREDICTIVE MODELS 6
   - 2.4 INCEPTION V3 ALGORITHM 7
   - 2.5 DATA ACQUISITION 9

3. PREDICTION MODEL ARCHITECTURE 15
   - 3.1 STATISTICAL MODELS 15
   - 3.2 MACHINE LEARNING 19
   - 3.3 RECURRENT NEURAL NETWORKS 20
   - 3.4 TRANSFORMER NEURAL NETWORK 22
   - 3.5 FEATURE-BASED FORECAST MODELING AVERAGE (FFORMA) 26

4. RESULTS 28
   - 4.1 STATISTICAL MODEL RESULTS 28
   - 4.2 RECURRENT NEURAL NETWORK RESULTS 40
   - 4.3 TRANSFORMER NEURAL NETWORK RESULTS 59
   - 4.4 FFORMA RESULTS 66
   - 4.5 RESULT DISCUSSIONS 70

5. CONCLUSIONS AND FUTURE WORK 73

REFERENCES 75

VITA 84

LIST OF TABLES

Table Page

1. Univariate GPU utilization RNN model prediction evaluation metrics 43
2. Univariate power draw RNN prediction evaluation metrics..... 47
3. Multivariate GPU utilization RNN prediction evaluation metric 52
4. Multivariate power draw RNN prediction evaluation metrics 56
5. GPU utilization Probabilistic TNN prediction results 61
6. Power draw Probabilistic TNN prediction results 62
7. Annotated GPU utilization prediction metric summary..... 64
8. Power draw Annotated TNN prediction evaluation metric results..... 65
9. GPU utilization FFORMA prediction evaluation metrics 67
10. FFORMA power draw prediction evaluation metrics..... 70
11. GPU utilization prediction model summary..... 71
12. Power draw prediction model summary..... 72

LIST OF FIGURES

Figure Page

1. Inception v3 algorithm architecture 8
2. Trace data for GPU utilization and power draw 10
3. Strong scaling test 13
4. Weak scaling test 14
5. Rolling prediction on a horizon method 18
6. SimpleRNN cell architecture 21
7. LSTM cell architecture 22
8. TNN architecture 23
9. FFORMA architecture 27
10. GPU utilization stationary trace 29
11. ACF and PACF GPU utilization 30
12. GPU utilization MA(2) prediction results 30
13. GPU utilization AR(4) prediction results 31
14. GPU utilization ARMA(4, 2) prediction results 32
15. Power draw stationary trace 33
16. ACF and PACF power draw 34
17. Power draw MA(1) prediction results 34
18. Power draw AR(16) prediction results 35
19. Power draw ARMA(16, 1) prediction results 36
20. GPU utilization Auto-ARIMA prediction results 38
21. Power draw Auto-ARIMA prediction results 38
22. GPU utilization non-seasonal Auto-ARIMA prediction results 39
23. Power draw non-seasonal Auto-ARIMA prediction results 40

Figure Page

24. GPU utilization RNN prediction results with lowest test MSE and RMSE 44

25. GPU utilization RNN prediction results with lowest test MAE 44

26. GPU utilization RNN prediction results with lowest test Within 2.5%..... 45

27. Prediction results GPU utilization RNN with lowest test Within 1.0% 45

28. Prediction results power draw RNN with lowest test MSE and RMSE 48

29. Prediction results power draw RNN with lowest test MAE metric..... 49

30. Prediction results power draw RNN with lowest test W2.5 metric..... 49

31. Prediction results power draw RNN with lowest test W1.0 metric..... 50

32. Multivariate GPU utilization prediction results producing lowest test MSE and RMSE..... 53

33. Multivariate GPU utilization prediction results producing lowest test MAE..... 53

34. Multivariate GPU utilization prediction results producing lowest test W2.5 and W1.0. 54

35. Multivariate power draw prediction results producing lowest test MSE and RMSE. 57

36. Multivariate power draw prediction results producing lowest test MAE..... 57

37. Multivariate power draw prediction results producing lowest test W2.5..... 58

38. Multivariate power draw prediction results producing lowest test W1.0..... 58

39. GPU utilization Probabilistic TNN prediction results and metrics per trace 61

40. Power draw Probabilistic TNN prediction results and metrics per trace 63

41. GPU utilization Annotated TNN prediction results 64

42. Power draw Annotated TNN prediction results 66

43. GPU utilization FFORMA prediction results..... 67

44. GPU utilization FFORMA prediction results..... 68

45. Power draw evaluation prediction results..... 69

46. FFOMRA Auto-ARIMA weights 70

CHAPTER 1

INTRODUCTION

In this digital era, modern computing systems are becoming unable to keep up with the demand for computationally expensive and memory-intensive applications. Usually, the programmer is required to sacrifice accuracy or performance in favor of runtime. There is an increase in demand in particular for Graphics Processing Units (GPUs), that specialize in solving mathematical computations used in the training and inference of computationally intensive models.

1.1 MOTIVATION

This work was done to leverage predictive algorithms for applications, such as machine learning, to investigate the runtime performance of the application *Inception v3* (or "Inception3") [1]. Runtime performance for the application Inception3 includes investigations of parallel scaling. Other investigations include two different performance metrics that provide insight into how an application executes. The two performance metrics are GPU utilization, measured in percent, and power draw, measured in Watts. Algorithms that produce accurate predictions can be leveraged to affect the efficiency of an algorithm. Specifically, this work makes progress towards utilizing accurate predictions on GPU utilization and power draw to provide energy savings.

Predictive algorithms specialize in solving a particular type of problem. There are two main categories of prediction focused herein. The first is statistical models and the second relies on machine learning. The prior will serve as a baseline for comparing the results of the former. The choice of prediction model often is data and domain-specific. One data-specific category specializes in predicting time series data. Time series predictive algorithms assume that the data is input in a sequential temporal order. This temporal order lists data at regular intervals of time. Time series predictive algorithms use a certain number of past values to predict future values.

One of the goals beyond the current scope of this work was to implement online learning algorithms. "Online learning" is a category of machine learning algorithms that learn on continu-

ous streams of sequences of data (i.e., in real-time, as the data is being collected). This contrasts with "offline learning," which trains on the entire training data set at once [2].

1.2 OBJECTIVES

The objective was to study various time series prediction algorithms and study the accuracy of their prediction results on GPU utilization and power draw traces of GPU parallel *TensorFlow 1.15* implemented *Inception v3* (or "*Inception3*") application. The trace data consists of two application performance metrics, GPU utilization and power draw, recorded during two runs of an intensive parallel image classification algorithm *Inception v3*. This work studied and implemented various time series predictive models, such as statistical-based models (SARIMA), Recurrent Neural Networks (RNN), Transformer Neural Networks (TNN), and Feature-Based Forecast Modeling Average (FFORMA). The objective is that the chosen model's predictions in the future can be leveraged in a run-time environment to provide energy efficiency. In addition, time-consuming predictive algorithms were explored since the trace data was similar between different traces for a particular GPU. This work focuses on the trace data from GPU 1.

1.3 RELATED WORKS

Significant decreases in temperature and power draw can be accomplished by implementing power-capping on GPUs at a supercomputing center with minimal effects on job performance [3]. Prior work by [3] studied the effects of reducing peak power to 60% for a pre-trained BERT model, resulting in an increased computational time (by 8.5%) but with a 12.5% energy savings [4]. In their more recent paper, BERT training under a 200W power cap resulted in a 15% reduction with little degradation in training speed [3]. Reducing power consumption by implementing a frequency cap is promising, but results in unconventional and unpredictable execution time [5]. Experimental results aiming to minimize the training time by utilizing a cooperative distributed GPU power capping system for GPU-based clusters found that it improves power capping accuracy to have a mean absolute error of less than 1% [6].

A related field for forecasting energy is energy consumption by devices. An enhanced Neural Basis Expansion Analysis for Interpretable Time Series (N-BEATS) saw success in energy consumption predictions for smart grids and smart homes [7]. A study on forecasting energy consumption of buildings revealed that a multivariate Transformer Neural Network performed 3.2 percentage points better than RNN models in terms of MSE and MAPE evaluation metrics [8].

Reference [9] surveyed predicting power and the following works were discussed. Song et al. used a neural network with an input layer size of 10 and two hidden layers of size 4 in conjunction with a counter-based performance model. This model was more accurate than linear regression models. "Performance counters" utilize hardware actions, such as average time per operation. Chen et al.'s "GPU Power Model built on GPGPU-Sim Simulated Events" compared linear regression, regressive tree, and random forest models, showing that random forest performed best for estimating kernel energy. Ma et al. found that a support vector regression model outperformed a support linear regression model.

In Reference [10], experimental transformer neural network results on influenza-like illness time series data outperformed the LSTM model. The transformer model resulted in a relative RMSE decrease of 27% [10]. A study into power profiling of Transformer Neural Networks for text translation tasks, revealed that PyTorch (as opposed to Numpy's) implementation resulted in shorter runtime, and thus lower cumulative energy consumption [11]. Reference [12] studies power consumption with different GPU machine configurations. In this work, the best base model, with respect to MAE, was *SMOReg* (a model that finds a curve to map inputs to outputs). Comparing this model with the most optimal ensemble model, that combined the results of three different models, reduced the MAE from 4.5% to 3.5%.

Research into energy efficiency through power limiting focuses mostly on CPUs rather than GPUs. Predicting GPU utilization was previously studied by [13] but for Deep Learning frameworks in the cloud. The research utilized a predictive engine that extracted metrics from its model computation graph. It achieved up to a 61.5% improvement in GPU cluster utilization. In [14], an algorithm was proposed that would predict and based on those prediction values, change

the maximum allotted power at runtime. The paper reported the highest average energy savings of 17.7% with the lowest average performance loss of 5.1%. In [15], the continuance of [14], a similar approach was taken, GPU utilization was modeled as a time series and an Autoregressive Integrated Moving Average (ARIMA) process was used to dynamically predict the GPU utilization in the next timeslice to be executed. This paper utilized the predictions in each GPU to allocate power in accordance with the predicted utilization.

1.4 THESIS ORGANIZATION

The remaining chapters of this thesis are organized as follows. Chapter 2 explains the background and describes relevant topics. Chapter 3 presents the architecture of the predictive models. Chapter 4 presents, analyzes, and discusses the prediction results. Finally, Chapter 5 presents the results of this research and discusses future work proposed.

CHAPTER 2

BACKGROUND

2.1 POWER CONSUMPTION

There are two main classes of power consumption for GPUs, leakage of power and dynamic power. Leakage of power occurs when a GPU is powered (but not actively being used). Dynamic power is consumed based on runtime and it is a function of circuit technology and operating temperature [16]. A megawatt of power costs about one million dollars per year. To put that in perspective, the top-ranking supercomputer for November 2022's Green500 was Henri, which showed an energy efficiency of 65.09 GFlops/Watts [17]. This same report took note of Frontier, a supercomputer with a good balance between performance and energy. Its energy efficiency was 52.2 GFlops/Watts with a Rmax (realistic max performance) value of 1,102 PFlops/second. This was accomplished due in part to four purpose-built AMD Instinct 250X GPUs [17] [18].

Energy efficiency is important to both decrease costs and decrease the impact on the climate. Applying energy efficiencies comes with trade-offs in performance. However, these trade-offs can be minimized. One way to apply model efficiency, and thus energy efficiency, is by training on smaller representative data. However, this does require specialized domain knowledge [3].

2.2 PERFORMANCE MEASURES

Collecting performance measures on an application provides further insights about its performance that can lead to investigations into implementing further optimizations. After understanding how an algorithm executes, metrics can track the improvements, or lack thereof of the proposed optimizations implemented. *nvidia-smi* [19] is a light-weight utility that collects performance measures on parallel GPUs, such as memory, GPU utilization, power draw, and GPU temperature [19]. When picking an appropriate performance measuring tool, it is important to consider any implications on the performance of the application.

Nvidia-smi provides a basic overview of the performance statistics for each parallel GPU while minimizing effects on the performance [19]. Other performance profilers such as DL-Prof, PyProf, and NVIDIA Nsight Systems profiler provide more detailed statistics, such as wall GPU idle percentage and tensor core kernel efficiency that are not necessary to collect given the scope of this work, unnecessarily increasing memory and GPU utilization [20]. With an already memory-intensive algorithm, using a profiler that requires memory would increase and introduce the possibility of a bottleneck that would skew the true performance values. Hence, collecting GPU utilization and power draw data herein employed nvidia-smi.

2.3 TIME SERIES PREDICTIVE MODELS

Forecasting models attempt to predict future values based on learned historical data. There is a subtle difference between forecasting and predicting. Predicting models temporarily removes a portion of the data to evaluate the model's predictions against. Forecasting models attempt to predict values beyond the data set.

For time series prediction models, the original time series data ("trace") is split into different subsets. For statistical models, these data sets are split into data to fit on and data to compare against the model's prediction. For machine learning models, these data sets are split into train, validation, and test sets. The train data set consists of data to be learned. Training on this data set is the most expensive section of code (both in terms of memory and time). During training, most algorithms utilize linear algebra to learn the training data set. The validation data set is used to check the learning ability of the model on unseen data. The test data set is used to evaluate the final model's prediction ability on data not shown to the model.

2.3.1 Evaluation Metrics

Comparing evaluation metrics provides insight into which model predicts the most accurately. Accuracy equates to how close a model's predictions are to the actual value. Evaluation metrics were collected on the different prediction models. The main two evaluation metrics col-

lected were mean absolute error (*MAE*) and mean square error (*MSE*), defined as follows,

$$MAE = \frac{1}{N}\sum_{n=1}^{N}|p_n - \hat{p}_n| \qquad and \qquad MSE = \frac{1}{N}\sum_{n=1}^{N}(p_n - \hat{p}_n)^2 \ , \tag{1}$$

where p_n is the ground truth (i.e., actual) and $\hat{p}_n$ is the predictions [21]. Other evaluation metrics collected on predictive algorithms included root mean squared error (*RMSE*), the square root of the MSE metric. Another metric is symmetric mean absolute percentage error (*sMAPE*), a symmetric error type that compares ratios of errors, defined as follows,

$$RMSE = \sqrt{\frac{1}{N}\sum_{n=1}^{N}(p_n - \hat{p}_n)^2} \qquad and \qquad sMAPE = \frac{1}{N}\sum_{n=1}^{N}\frac{|p_n - \hat{p}_n|}{p_n + \hat{p}_n} \ , \tag{2}$$

where p_n is the ground truth (i.e., actual) and $\hat{p}_n$ is the predictions [21].

Two other metrics considered were *Within 2.5%* (abbreviated to *W2.5*) and *Within 1.0%* (abbreviated to *W1.0*). These metrics take the percentage of the prediction values in the data set that are within a relative tolerance amount (2.5% and 1.0%). The optimal value is 100%, indicating that all of the predictions were within the relative tolerance amount for the actual values contained in the data set.

2.4 INCEPTION V3 ALGORITHM

The "application," or "algorithm," being investigated is Inception3. As explained in more detail below in Section 2.5, the performance measures, GPU utilization and power draw, constitute the trace. Inception3 is an image classification model that utilizes convolution neural networks (CNN). The architecture of this model utilizes many convolution layers and a max pooling, linear, and softmax layer as shown in Figure 1.

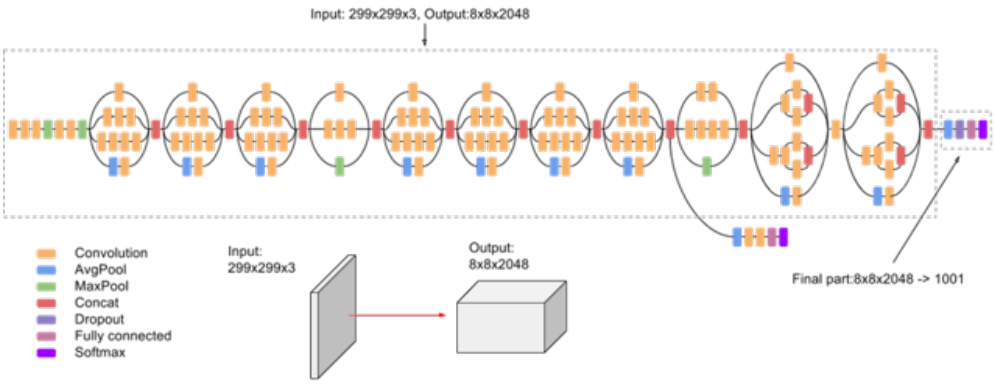

Figure 1. Inception v3 algorithm architecture.

2.4.1 Parallelism of Inceptionv3

The parallelization of this algorithm was mainly handled by *tf_cnn_benchmarks* [22]. The *variable_mgr* class sets how the parallelization of the variables and gradients are implemented. It defines whether each GPU has its own set of variables and how the gradient information is passed, or synchronized between GPUs. The trace data used a replicated mode, where each parallel GPU has its (own) copy of the variables. Gradients are combined and then replicated on all GPUs using an all-reduce or other cross-device aggregation. Gradients are used in the training portion of the algorithm to calculate the loss.

Inception3 employs an optimization - XLA (Accelerated Linear Algebra), which applies auto-jitter, enabling the TensorFlow's (TensorFlow is a python machine learning platform) algorithms to run faster. Using XLA is very useful for linear algebra-intensive applications. XLA combines and computes linear algebra-related tasks on a single GPU kernel. The biggest improvement that XLA provides relates to the memory bandwidth. Instead of writing intermediate computations to memory, it "streams" the information [23].

The all-reduce function utilized NVIDIA Collective Communication Library (NCCL) that provides routines for multi-GPU and multi-node communications, such as all-reduce and broadcast. This tool optimizes communication to achieve high bandwidth and low latency over

internodal connections – via PCIe and NVLIink. The documentation boasts a “2x peak bandwidth with in-network all reduce operations” [24].

The application code also optimizes parallelism by automatically setting the number of “global” batches as the product of the number of batches per GPU and the number of GPUs, both are provided by the user. This number of batches multiplied by the batch size (100) results in the number of epochs. Making the number of epochs a function of the number of GPUs assists in parallel scaling during training.

2.5 DATA ACQUISITION

Lambda Labs [25] provides a suite of image classification algorithms implemented in parallel with an option to run them on CPUs and GPUs [1]. Inception3 trained on synthetic data, created using random pixel colors generated on the GPU memory [22]. Along with the code, Lambda Labs provides benchmarks, throughput, as images per second to establish the relative effectiveness of GPUs [22] [25].

2.5.1 Trace Data

The data collected for this paper consists of two performance measures collected during calls to the *Inception3* application. *Inception3* was run using TensorFlow 1.15. The trace data consists of both performance measures, GPU utilization [%] and power draw [Watts], across multiple GPUs. GPU utilization corresponds to how busy each GPU is over time. More specifically, the percentage “of time over the past sample period during which one or more kernels was executing on the GPU” [19]. Power draw is measured in Watts (W) and is monitored across the entire board (accuracy being within 5 Watts) [19]. Measurements were recorded every 250 milliseconds. For now, GPU 1 was the focus. Due to the rest period (10 seconds) between the runs of the algorithm, any portion with more than 10 seconds of consecutive 0% GPU utilization was removed [22]. This explains why the x-axis for the visualizations herein is time steps [250 ms]. Scaling was performed on the input trace except otherwise specified using MinMaxScaler.

MinMaxScaler shifts and scales the data so that the data ranges from zero to one [26]. The MinMaxScaler retains the original shape of the data set. Scaling keeps the model weight values low. Without it, some model can suffer from poor performance during training [27]. However, that is not true with all types of models.

Each trace consists of two calls of the Inception3 application see Figure 2. In total, Inception3 was run four times. The first two have 1568 batches and 0.12 epochs, and the final two have 3072 batches and 0.24 epochs. The batch size relates to the amount of synthetic data and the number of epochs correspond to the number of times that the algorithm trains on that data. The first two resulted in around 2,200 images per second and the last two resulted in around 6,200 images per second [22].

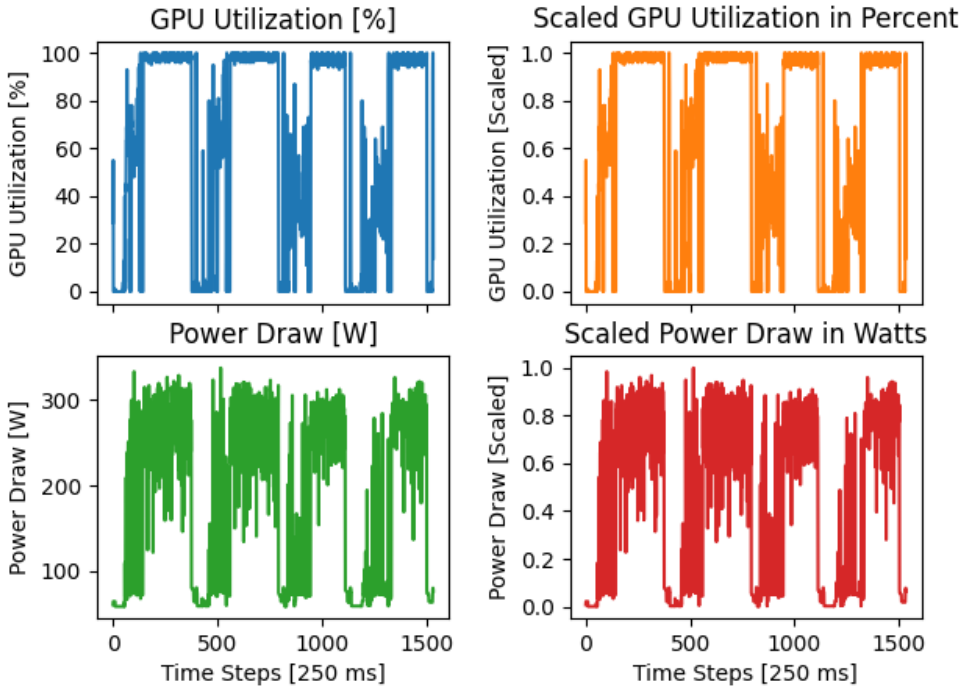

Figure 2. Typical output for the GPU utilization and power draw for GPU 1 both in the original units (on the left side) and after using MinMaxScaler [26] (on the right side).

The 250-milliseconds (ms) interval and regularity of the linear algebra operations within the training portion of the Inception3 architecture makes the trace data good time series candidates. The recording interval was 250 ms because it provided the appropriate level of granularity.

2.5.2 Computing Platform

Data was acquired from a computational node DGX-1, equipped with eight Xeon® - Tesla V100 SXM2-32 GB Graphic Processing Units (GPUs). The link speed of the NVlink is 25.781 GB/s. The maximum power is 300 Watts and the minimum is 150 Watts for each GPU. Data was collected across all eight parallel GPUs. The clock rate was 1.53 GHz. There are five cores per GPU, so 40 total.

2.5.3 Parallel Scaling

Implementing thoughtful parallelism can introduce energy savings to an algorithm. For parallel algorithms, some portions of code must be performed in serial (by one processor). The time it takes to execute these portions will not change when increasing the number of computational resources available during the parallel portions of code. Scalability measures how much throughput benefits are gained when additional processors are added [28].

Two different types of tests are involved in analyzing the algorithm's parallel efficiency (also called scaling). They investigate how the speedup or throughput of the algorithm is affected by the amount and how the work is distributed among various computational resources. Throughput is the amount of work done in a particular amount of time [28]. In this case, throughput is the number of images per second.

The first type of test is strong scaling. For this type of analysis, the problem size is held constant, while the number of workers increases. This means that as the number of workers increases, the amount of work per worker decreases [28]. The global batch size is kept constant, while the batches per device decrease. The second type of test is weak scaling. For this type of analysis, the number of workers increases and the work per device is held constant. This means

that both the number of workers and the problem size increase [28].

The framework provided by TensorFlow used to implement the Inception3 model allowed the user to specify the number of parallel GPUs. By default, the algorithm does not allow work to be shared among cores or threads [22]. Therefore, the level of granularity used for the number of workers is equivalent to the number of GPUs. The amount of batches per GPU was set in a configuration file. Parallel scaling analysis was done on the throughput during both the training portion and when evaluating the model (on the validation data set).

For strong scaling during both training and evaluation, the number of batches was set to 360, 480, and 600 and the number of GPUs was set to 2, 4, 5, 6, and 8. Due to out-of-memory error, the number of GPUs for the global batch size was set to 3, 4, 5, 6, and 8 when the batch size was 600. The results for both the training and evaluation are visually identical, so only the strong scaling results from the evaluation portion are shown in Figure 3. Strong scaling analysis asks how increasing the amount of computational resources affects the parallel performance. Given a large global batch size, there will be a point where increasing the number of GPUs to solve the same problem will cause a degradation in performance. Strong scaling was less than the ideal linear throughput scaling, which is achieved if all of the parallel portions of code are ideally parallelized. The scaling of throughput implies that decreasing the global batch size lower than 480 causes more significant parallel inefficiencies as more GPUs are added.

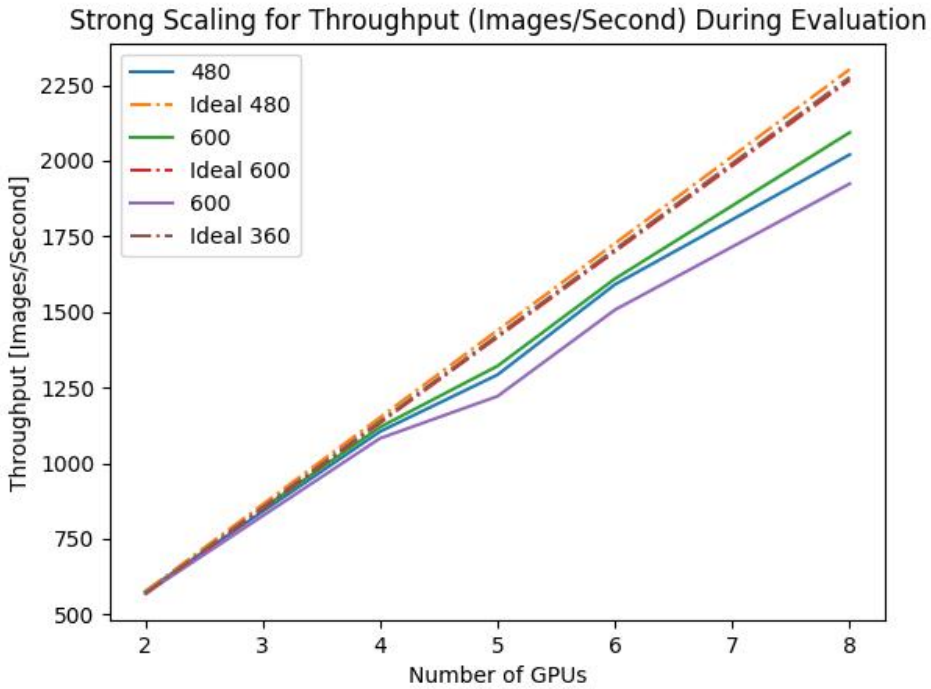

Figure 3. Strong scaling with various GPU counts with a constant batch size of 360, 480, and 600 and with throughput measured during evaluation. Throughput is less than the ideal linear scaling.

For weak scaling during both training and evaluation, the number of batches *per device* was set to 196, 225, and 384 and the number of GPUs was set to 2, 4, 5, 6, and 8. The results for both the training and evaluation are visually identical, so only the weak scaling results from the evaluation portion are shown in Figure 4. Weak scaling was close to the ideal linear throughput scaling, which is achieved if all of the parallel portions of code are ideally parallelized. As the number of batches *per device* increases, the further it deviates from the ideal linear speedup. This trend is typical. As the number of batches *per device* increases, the higher the potential for uneven workloads, leading to more deviations from ideal parallelism. The results of the weak scaling demonstrate that the code [22] implements a good parallel framework that balances larger workloads well.

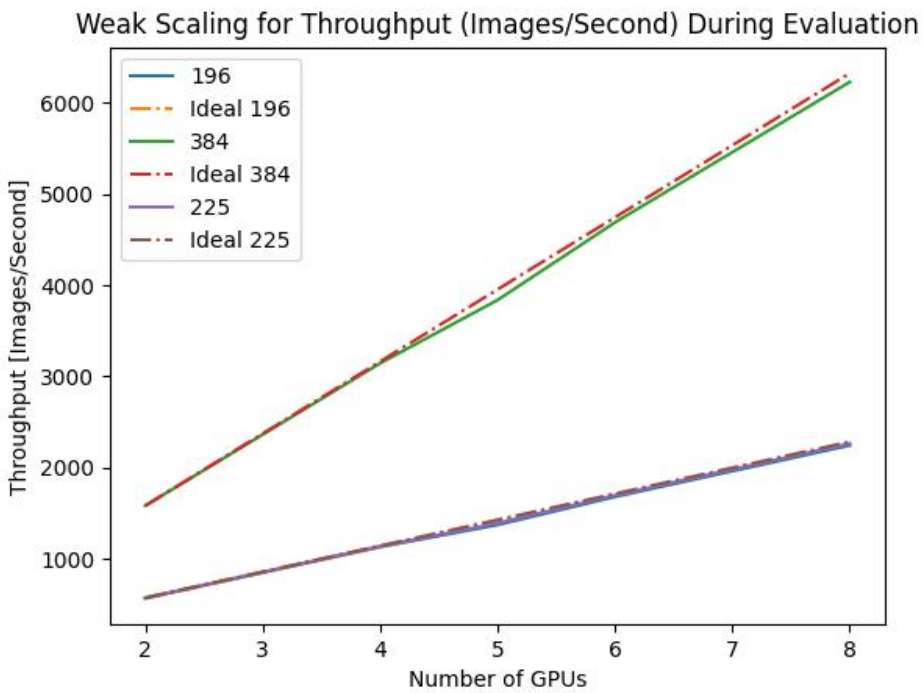

Figure 4. Weak scaling with 2, 4, 5, 6, 8 GPUs with a constant batch *per device* size of 196, 225, and 384, with throughput measured during evaluation. There is a nearly linear throughput scaling for the given batches *per device*.

The dedicated DGX-1 node GPUs were the ideal number, since the algorithm does not scale well beyond eight GPUs (due mainly to workload imbalance). Also, time and power are wasted on idle cores and synchronization [16] [22].

To best match the benchmarks provided by [25] for the Tesla V100 SXM2-16GB, the following image settings were replicated. The code called TensorFlow 1.15 Inception3 model on synthetic data. Since the benchmark code results from [25] utilized 16GB GPUs instead of the 32GB ones, the number of images per second should be larger for the DGX-1 node.

CHAPTER 3

PREDICTION MODEL ARCHITECTURE

3.1 STATISTICAL MODELS

Predictive statistical time series models use historical behavior to explain current or future behavior. Predictive time series statistical models rely on time series analysis, and its ability to create mathematical models for stationary time series sequences. These models are developed from patterns or characteristics in the data, such as trends and seasonal patterns. The assumption for stationary time series is that the mean and variance are constant at regular periodic intervals. Stationary time series also have an autocorrelation that depends on the distance between two-time steps relative to a regular periodic interval. In other words, the relationship between periods does not change over time. Weaker relationships produce autocorrelation values closer to zero. For a time series to be considered stationary, the autocorrelation should be significantly different than zero. Making the data stationary makes predicting easier, since the mean, variance, and autocorrelation do not change over time. Without the assumption that the trace data is a stationary time series, forecasting using statistical models would be unreliable [29].

To deduce the appropriate statistical prediction model, use the following decision sequence explained in Section 3.1.1, as suggested in [29]. Note that the models constructed are non-seasonal, meaning that they do not consider seasonal effects. There are seasonal equivalences of the models that predict using multiples of the given seasonal period.

3.1.1 Statistical Trace Analysis

Stationary Tests

First, determine whether the data is stationary by using different stationary tests. The most popular is Augmented Dickey-Fuller (ADF) test. This test finds the presence of a unit root

to determine whether the data is stationary. Once the data is stationary, the augmented Dickey-Fuller (ADF) test results in a p-value that is below a certain significance level of 0.01 (the more common is 0.05). The test statistic should be a large negative number and must be less than the critical value of the ADF test. If these conditions are true, it provides sufficient evidence to reject the null hypothesis that the series is non-stationary [29]. The other popular stationary test is the Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test. For this test, the null hypothesis is switched. Meaning, for the test to conclude that the times series is stationary, the p-value must be greater than the significance level of 0.01 and the test statistic should be less than the critical values from the test [29]. The failing of a stationary test necessitates a transformation of the data to achieve a stationary time series sequence.

The transformation to apply depends on which stationary test fails. If the results for both ADF and KPSS determine that the series is not stationary, then the series is not stationary. If both test results conclude that the series is stationary, then the series is stationary. If the KPSS test results conclude that the series is stationary, but the ADF does not, then transform the data by removing the trend ("detrending"). If the ADF test results conclude that the series is non-stationary, but the ADF test does not, then apply a differencing [30].

A differencing of d applied to a time series shifts the original trace back d number of periods. This differencing can be noted with a backwards shift operator B. A differencing of one is written as $y' = y_t - y_{t-1} = y_t - By_t = (1-B)y_t$, where y' is the transformed time step, y_t is the data at time t and $(1-B)$ represents a first differencing. Applying a k order differencing is written as $B^k Y_t = Y_{t-k}$ [29] [31].

Random Walk Autocorrelation Check

After transforming the trace data into a stationary time series, plot the autocorrelation plot (ACF) that displays the autocorrelation and lags. This plot is used to determine if the stationary time series is a random walk. It is a random walk if there are no significant autocorrelation coefficients significantly different than zero after lag zero [29].

Constructing MA, AR, and ARMA Models

If the series is not a random walk, the lags from the ACF plot can be used to determine the order for the moving average (*MA*) model. Look at the ACF plot and determine the consecutive lag value right before the autocorrelation coefficients start becoming non-significantly different than zero. This lag value is referred to as lag q and it is used to construct a MA(q) model [29]. The model's θ weights are memory coefficients, where θ_{t-k} represents the amount of influence that the data k steps away has on the current data. The $MA(q)$ model structure is

$$X_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + ... + \theta_q \varepsilon_{t-q}, \tag{3}$$

where the current data at time t is the white noise ε_t added to the sum of the product of past forecasting errors $\varepsilon_t - k$ and their associated influence θ_{t-k} for $k = 1, 2, 3, ..., q$ [31].

To construct an autoregressive model, repeat the earlier steps to transform the original trace into a stationary series. Remember to verify it is not a random walk after finding the stationary series. This time, instead of looking at the ACF plot, look at the PACF plot. The lag value right before the partial autocorrelation coefficient starts becoming non-significantly different than zero is order p. This lag value can be used to construct an AR(p) model [29]. The model's error shock $\phi_1^k \varepsilon_{t-k}$ is the amount of influence that the data at $t-k$ has on the current time step t. The coefficients in the equation are referred to as memory coefficients or ϕ weights. As the distance between the current time step at t and k increases, the amount of influence decreases. The $AR(p)$ model structure is

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + ... + \phi_p X_{t-p} + \varepsilon_t, \tag{4}$$

the data at time t, X_t, is the sum of the error shocks multiplied by its corresponding data [31].

To construct an autoregressive moving average model (*ARMA*) model, use the orders p and q found using the instructions above. An ARMA model is a combination of autoregressive and moving average models and is written as *ARMA(p, q)*. The $ARMA(p,q)$ is modeled as

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + ... + \phi_p X_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + ... + \theta_q \varepsilon_{t-q}, \tag{5}$$

where the data at time t is the sum of the linear combination of past values (with their associated errors) (AR) and the linear combination of the past forecasting errors (MA) [10] [31].

The MA, AR, and ARMA models fit on the fitted data set and predict the actual data set. This is done by utilizing a rolling forecast/prediction on a horizon method, see Figure 5. The fitted data set is increased by the window size w each iteration until the fitted data set is the same length as the entire trace. During each iteration, the fitted data set is used to predict the next w time steps [29]. The mathematical modeling came from [31] and [32].

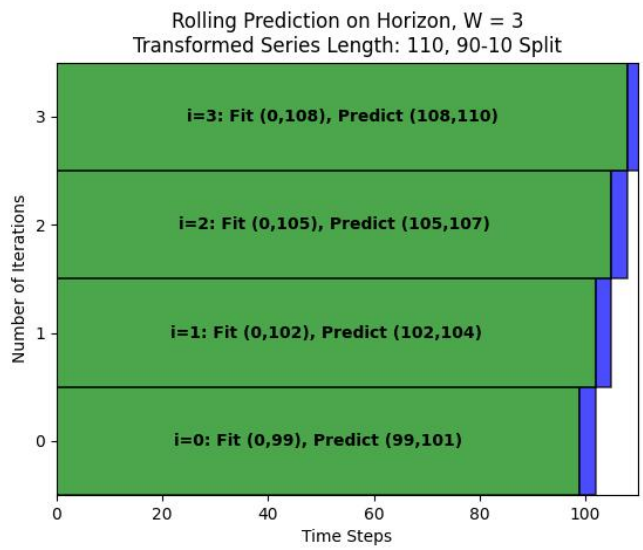

Figure 5. Rolling prediction on a horizon method, where the horizon is the length of the actual data set to predict. In this example, the horizon is 11. The model fits on the fitted data set and predicts the next w time steps. With each iteration, the fitted data set grows by w.

3.1.2 Auto-ARIMA

The ARMA model produced from the ACF and PACF plots does not consider any seasonal effects. The ARIMA model is an ARMA model that takes into account an integrated term of order d. This equates to the amount of differencing. A SARIMA model does consider seasonal

effects and is written as *SARIMA(p, d, q)(P, D, Q) [m]*. The orders *p, d, q* are explained in Section 3.1.1. The other orders, *P, D, Q* correspond to the orders for the seasonal version of the AR, seasonal integration order (i.e., seasonal differencing), and seasonal MA. Seasonal models introduce the idea that the trace can be stationary relative to a seasonal period *m* [29].

Since the prediction results depend on the given p, d, q, P, D, Q, and m, *pmdarima*, a Python library containing many time series analysis tools, provides a function *auto_arima* [33] that will try different order values and will return the "best" model that produces the lowest information criterion Akaike Information Criterion (AIC) score based on user-defined behaviors. The AIC score is defined as follows

$$2p - 2log(\hat{L}), \tag{6}$$

where p is the number of parameters, corresponding to the order number, $\hat{L}$ is the maximized value of the likelihood function [34].

3.2 MACHINE LEARNING

Unlike the previous statistical-based models, machine learning algorithms do not necessitate that the input trace data is stationary. Machine learning algorithms use neural networks that consist of multiple connected layers. Each of the layers consist of a defined amount of interconnected nodes that hold a weight value. Each layer applies a transformation using an activation function. Non-linear activation functions, such as *ReLU* and *Tanh*, enable the neural network to provide non-linear mappings between layers, and thus from the overall inputs to outputs [35]. Loss is a quantitative value used to evaluate the current model's ability to predict accurately. The optimizer adjusts the learned weights of the network to minimize the loss. However, neural networks are sometimes found to fail to identify abnormal patterns, and thus cannot fully map the inputs to the outputs [36].

One of the many choices a modeler must consider is the activation function when building a machine-learning model. Other considerations include the number of layers, the type of layer, the optimizer, and the number of hidden states for each layer. These considerations are

problem-dependent and can be difficult to combine to create the optimal machine learning prediction model. Since these considerations define the model, they will be herein referred to as "hyperparameters."

3.3 RECURRENT NEURAL NETWORKS

Recurrent Neural Networks (*RNN*) are powerful because this type of neural network assumes that the data provided is in a sequential order. It assumes that data temporally close are more correlated, referred to as short-term time dependencies. It implements this idea of contextual information through "hidden units" (or "hidden states") [37].

3.3.1 Simple Recurrent Neural Networks

The "hidden unit" values (or "hidden state" values) are stored in a state vector h_t at time step t. There are also weights associated with hidden units w_h that, in conjunction (through a dot product operation) with the hidden unit vector, provide most of the contextual information to the model. A bias for the hidden units b_h provides the remainder of the contextual information.

Each layer's inputs are the input and the previous state at time t. The output of that layer is the hidden units for the next time step, as shown in Figure 6. Therefore, for a fully connected RNN model, the final layer is a feed-forward layer (usually a Dense layer) [38]. Calculating the output y_t of a simple RNN layer (Equation 7) [37] requires an activation function [39]. The next state h_t also requires an activation function (Equation 7) [34].

$$y_t = activation(w_y \cdot h_t + b_y), \qquad h_{t+1} = activation(w_x \cdot x_t + w_t \cdot h_t + b_h) \tag{7}$$

The vanishing gradient problem is one of the main disadvantages of RNNs. This problem occurs with longer-length sequences. The neural network training will slow or become unstable, and, in some cases, stop [34] [40].

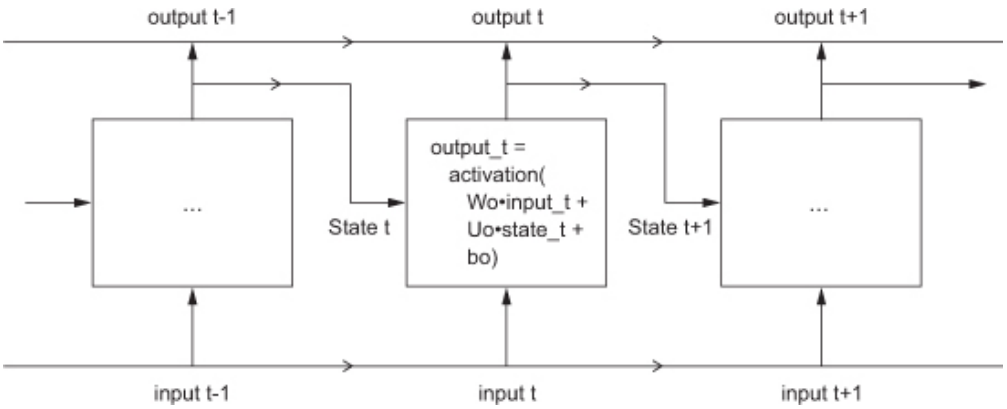

Figure 6. The architecture of a SimpleRNN cell [34]. The output is shown as the result of the activation function. Weights of the network are W_0 and $input_t$ is the input to the cell at time t. The $state_t$ is the hidden units. The bias vector is bo and it contains additional contextual information.

3.3.2 Long Short-Term Memory

Long Short-Term Memory ($LSTM$) networks are a specific type of RNN. LSTM layers attempt to remove the vanishing gradient effect by limiting the change gradient at each iteration. It assumes that longer-term contextual information affects future values. An LSTM cell, as shown in Figure 7, consists of three gates, each implementing an element-wise multiplication operation. The input gate controls what memory (i.e., data) will be added to the long-term state. The next type of gate, the forget gate, “forgets” some of the memories stored in the long-term state and replaces them with the memory selected from the input gate. Both the output of the forget gate and input gate are proceeded by an addition operation, producing the long-term state. The last type of gate is the output gate, which filters the results for the short-term state [34].

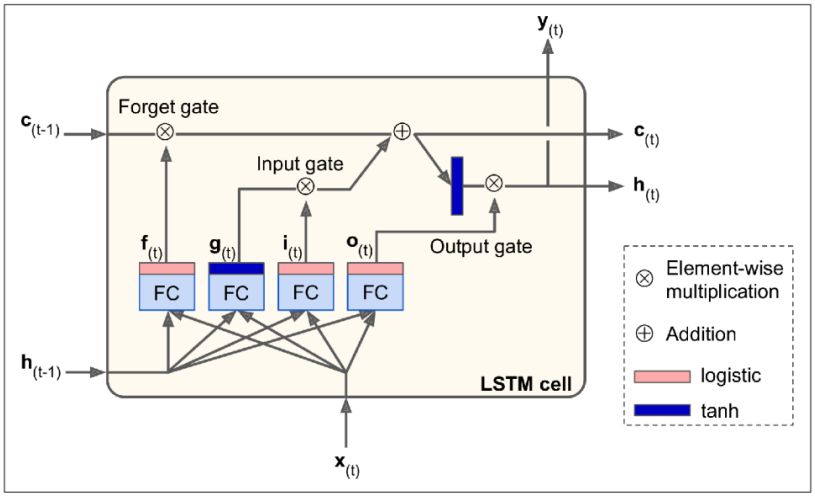

Figure 7. The architecture of a LSTM cell [39]. In this diagram, c represents the long-term state, and $h_{(t-1)}$ is the short-term state. The input $x_{(t)}$ and the short-term state are provided to four different fully connected (FC) layers. The main fully connected layer's output is $g_{(t)}$ and works the same as the RNN cell. The default activation function for an LSTM layer is $tanh$. f_t controls the forget gate. i_t controls the the input gate. o_t controls the output gate. The output is y_t.

There are several different types of LSTM models. The empirical results are discussed in the subsequent Section 4. Bidirectional LSTM networks add more time dependencies by extracting time dependencies in opposite directions [40]. An autoencoder decodes and encodes the information, which compresses (and optimizes) the input data and cell states [40]. The prior two LSTM models did not behave as well as the LSTM model with only LSTM layers.

3.4 TRANSFORMER NEURAL NETWORK

A Transformer Neural Network (*TNN*) is another type of machine learning algorithm used to perform time series prediction. TNNs rely on attention mechanisms implemented within the encoder and decoder layers. RNNs struggle with longer sequences and learning dependencies between elements from the input sequence that are temporarily farther apart [41]. Attention mechanisms (the mapping of queries, keys, and values to an output) solve this by creating global dependencies between inputs and outputs. However, there are limits to TNNs due to the quadratic compute and memory requirements with non-optimized self-attention [42]. The typical architec-

ture of a Transformer is explained and shown in Figure 8 as well as specific differences between the models used [43] below.

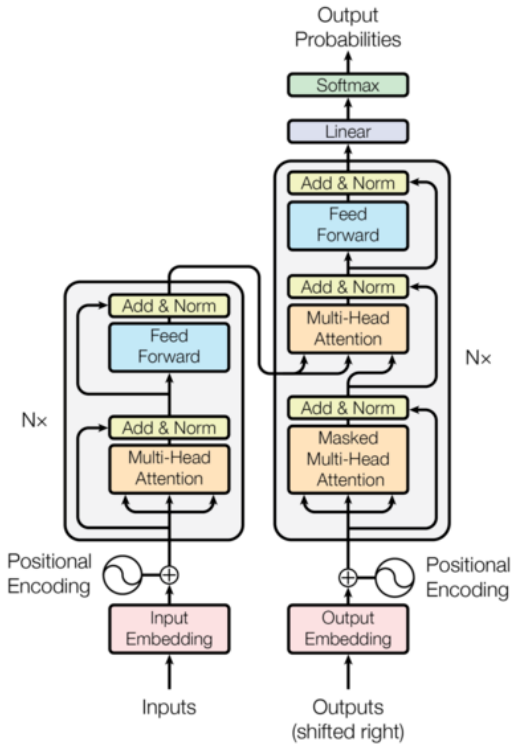

Figure 8. The architecture of a Transformer Neural Network [41] with the different layers and operations used. The Encoder section is on the left and is in grey. The *Nx* next to it signifies that the encoder consists of *N* stacked encoder layers. Similarly, the decoder layer is the grey section and the decoder consists of *N* stacked decoder layers.

The Input Embedding and Position Encoding layers encode the original sequence with positional information. The Encoder consists of identical encoder layers that are fully connected, stacked, and utilize self-attention. Each layer has two different sub-layers that employ multi-headed self-attention (which is, in parallel, learning information from different locations within the input sequence), and a position-wise fully connected feed-forward layer [41].

The output sequence is provided to the output embedding, positional encoding, and then to the decoder. The output embedding and positional encoding layers inject positional informa-

tion. The Decoder consists of identical decoder layers containing two multi-headed self-attention layers and a feed-forward layer. The input to the first multi-headed self-attention layer is either the output from the positional encoding (only for the first decoder layer) or the output from the previous decoder layer. The input to the second multi-headed self-attention layer is the output from the last encoder layer. The self-attention mechanism is employed in the decoder layers to prevent training on future data. Each sub-layer in both the encoder and decoder is followed by an add and normalize operation. The output of the last decoder layer is then transformed into a probability distribution through a linear layer followed by a Softmax layer [41]. An inference step (commonly a greedy search) is required to transform the output into a prediction sequence.

3.4.1 Probabilistic TNN

Reference [43] provides a probabilistic TNN time series forecasting model [43] that utilizes *Pytorch* [44], *Accelerate* [45] (for parallelization), and *GluonTS* [46] libraries. It trains a "global" probabilistic model using all available trace data (assumed to be time series data) and outputs predictions with an amount of uncertainty (+/- one standard deviation) [43]. These traces are split into train, validation, and test data sets according to the predetermined prediction length. For their corresponding data loaders, a batch is sampled from their data set and is created using an *InstanceSplitter* [47]. For the training and validation data loaders, it randomly selects a time series and a starting token that serves as the start of the sequence and extends it for the context length (double the prediction length) plus the prediction length plus the lags [43]. "Lags" provide additional data features and are produced using predetermined frequency information. The test data loader only samples the very last window [43].

The input to the encoder is past values. The output from the encoder is a Tuple of the last hidden state, with a shape of (batch size, sequence length, hidden size). The input to the decoder is future values. Additional features can be provided to the encoder and decoder, such as time features [43]. The Probabilistic TNN model [48] uses four encoder and decoder layers with a *d_model*, the number of dimensions of the data, of 32 [43]. Inference utilizes a Greedy Sam-

pling/Search to convert the output of the model back into predictions. A prediction distribution is created and utilizes an autoregressive sampler that is used to return the prediction outputs [43].

3.4.2 Annotated TNN

Another type of Transformer Neural Network was implemented. This model is implemented similarly to a typical transformer neural network and is optimized for classification [49] [50]. This type of model might provide interesting results when compared to one that is time series optimized. The input to the model was in integer form, where the integer value was treated similarly to a label. The data value was mapped to the positional location in the series, similar to models optimized for translations.

This TNN model loads in one trace. The trace is separated into a train and test data set. These data sets are split into batches of a particular forecasting horizon length. These batches can be distributed among parallel GPUs. For the model implemented for this work, the batches were distributed among three parallel GPUs. Each parallel GPU trains on its distributed train batches and evaluates on their distributed test batches.

For example, a model with a forecasting horizon length of 24 and an original trace of length 1535 is first split into a train and test data set. The train and test data sets are divided into batches, dropping data that prevents the batches from being on an equal length. The train data set consists of 42 sequences of length 24, equating to 14 batches per parallel GPU. The test data set consists of 21 sequences of length 24, equating to 7 batches per parallel GPU.

Additional modifications to the code allowed for an all-reduce averaging of the prediction evaluation metrics to be collected for each epoch. However, only the metrics from the last epoch are recorded.

The “mean” and “minimum” values for each of the models take the mean and minimum of the batches in the parallel GPU at the last epoch (after the model is trained). For this example, it would be the mean of the metric over the 7 batches in the test data set in the particular parallel GPU. The predicted and actual values for the batch resulting in the lowest MSE and the batch

resulting in the lowest MAE scores for the batches in the parallel GPU's test data set are saved.

3.5 FEATURE-BASED FORECAST MODELING AVERAGE (FFORMA)

The prior prediction models generated forecasts from a singular model or global model. Another type of machine learning prediction model combines forecasts from many different forecasting models into a singular "meta" model. This model performed well during the M4 competition (a machine-learning competition to create the best-performing model over 100,000 data sets from different domains) [51] [52]. Feature-based Forecast Modeling Average (*FFORMA*) combines the forecasts from many time series feature-based models using weights [52].

Instead of focusing on improving the performance of a singular prediction model, this model uses a weighted average of the forecasts from the pool of time series prediction models. The experimental results lead [52] to conclude that FFORMA can assign weights to forecasting methods specialized to the particular type of series [52].

3.5.1 FFORMA Model

The model [53] creates a train and test data set from the *reference set* of all the time series trace sequences. Each trace is split to create a training and test portion. The model trains on a portion of the data set using cross-validation by temporal holdout (the process of splitting the test portion of the trace) for training. However, predictions were conducted on all the time series in the original *reference set*. Time series analysis tools extract time series features, or characteristics, from the data. These features are input into the meta model and then forecasts are generated as well as their associated forecast errors. Each forecasting model is given a weight. When combined, they create a summary forecast loss that the model tries to minimize [52].

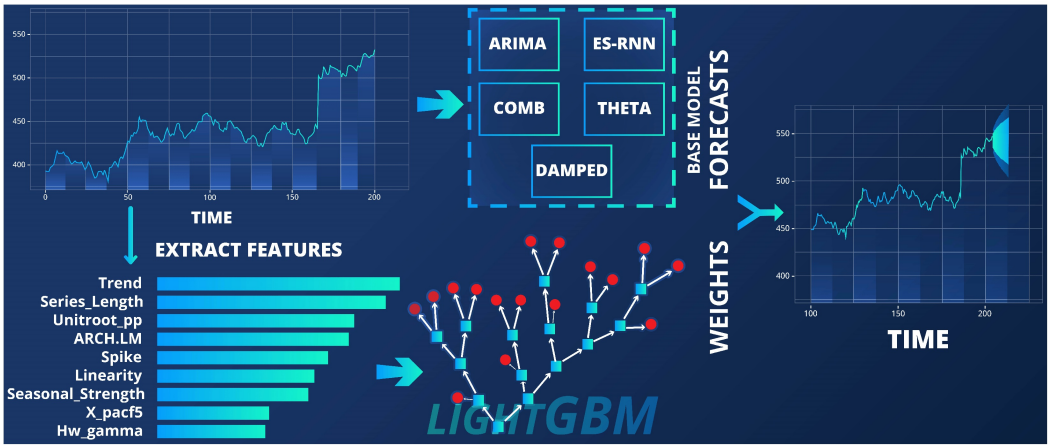

Figure 9. The FFORMA forecasting pipeline [54]. The features are extracted from the input time series and provided as inputs to the gradient boosting framework, LightGBD, that provides the weights. The pool of time series forecasting models produces forecasts from the input time series. The weights and the forecasts are used to provide the forecasts of the meta-model.

The time series features are extracted from the time series using *tsfeatures* [55], an R package containing many time series analysis tools [52]. The pool of forecasting models consisted of nine-time series prediction models, including auto-arima and neural network time series forecasts (nnetar), and extracted 42 features, including trend, series length, and uniroot-kpss [52]. FFORMA utilizes *xgboost*, which is a distributed gradient boost algorithm [52] [54]. The weights (that the optimizer adjusts based on the loss) represent the probability that the model produces the best predictions. A Bayesian optimizer with a limited search space finds the meta model's hyper-parameters [54]. When the input time series is not seasonal (is not given a period), the seasonal time series features are set to zero. The FFORMA forecasting pipeline is displayed in Figure 9.

CHAPTER 4

RESULTS

Comparison of the experimental results for the models utilized performance evaluation metrics as explained in Section 2.3.1. These performance evaluation metrics give insight into the accuracy of the model's predictions to the actual values in the test data set.

Experimental results were taken for both univariate and multivariate input data sets for the Recurrent Neural Network models. For the other models, only univariate input data was provided. "Univariate" and "multivariate" designate the format of the data provided to the model. Univariate model inputs are trace data restricted to a singular variable, either GPU utilization or power draw. Their output predictions are for the variable given. Multivariate models provide the trace data from both the GPU utilization and power draw.

4.1 STATISTICAL MODEL RESULTS

Establishing the statistical model results followed the decision sequence listed in Section 3.1.1. The data was split into a fitted and actual data set by dividing the trace into a 90-10 split. Time series analysis was performed on each trace. Because of the differences, the results from the trace data for GPU utilization are discussed first, then the power draw trace.

4.1.1 Statistical Model: GPU Utilization Results

Two stationary tests were performed. Section 3.1.1 discusses the conditions needed to conclude that a trace sequence is a stationary time series and how to interpret the ADF and KPSS results. The ADF test indicates that the GPU utilization trace is stationary, but is non-stationary according to the KPSS results. Therefore, a differencing is applied to the trace data. After a differencing was applied, both tests indicated that the differenced trace was stationary, so the differencing transformation resulted in a stationary time series. Figure 10 displays the stationary time series. The lag after which the autocorrelation coefficients become non-significantly different than zero is two according to the ACF plot as shown in Figure 11, so $q = 2$. Since $q > 0$, the stationary trace is not a random walk. Figure 12 displays the actual and prediction for the $MA(2)$ model. After reversing the differenced predictions, the model resulted in an MSE of 2450.08, MAE of 26.09, and RMSE of 49.50.

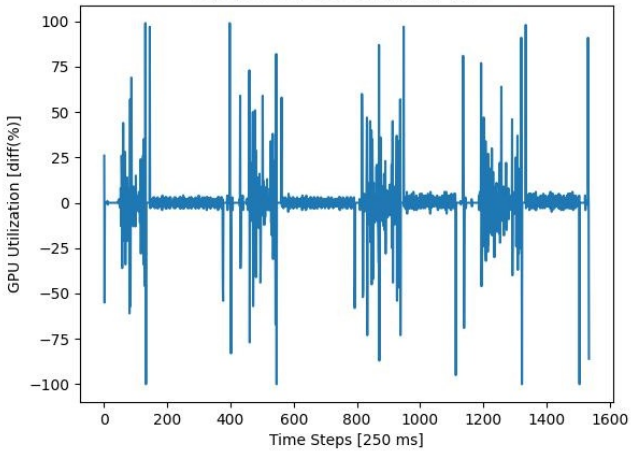

Figure 10. GPU utilization stationary trace created from applying a differencing transformation.

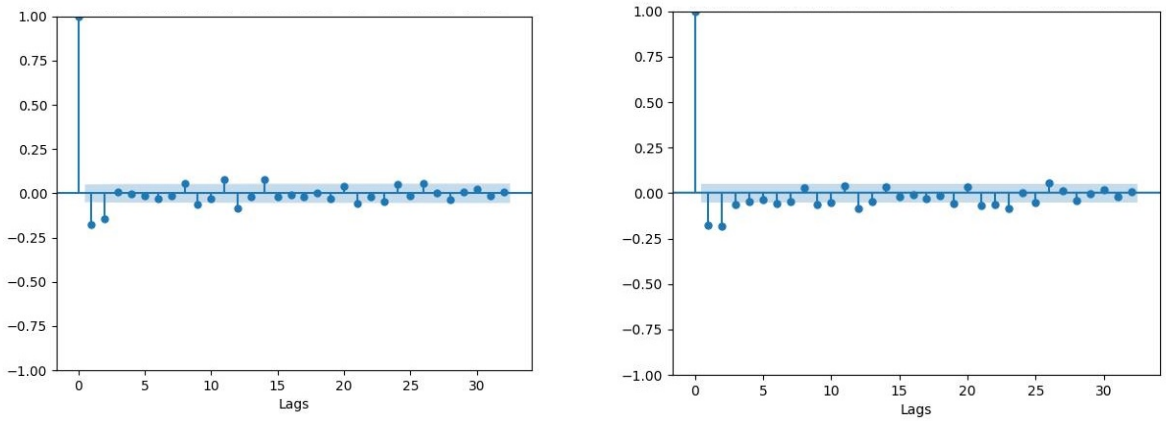

Figure 11. Left, an ACF plot for the differenced GPU utilization trace. Lags are significantly different than zero consecutively until lag two, so $q = 2$. The shaded area represents where the autocorrelation coefficients are not significantly different than zero. Right: a PACF plot for the differenced GPU utilization trace. Lags are significantly different than zero until lag 4, so, $p = 4$.

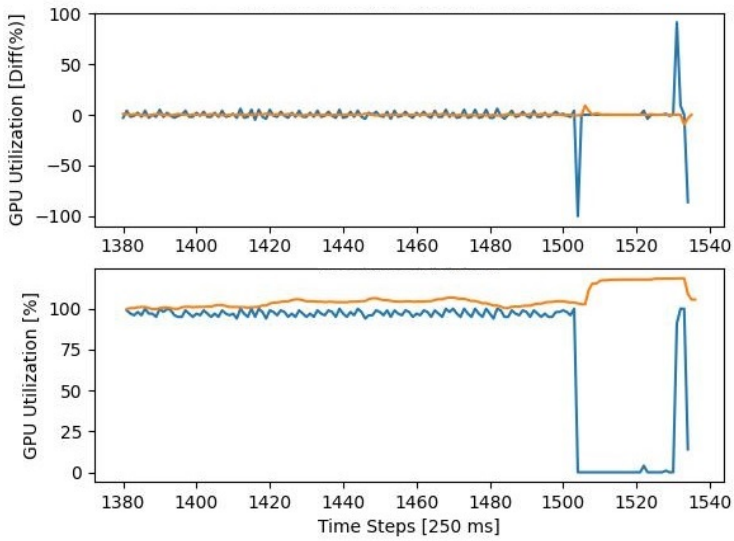

Figure 12. GPU utilization MA(2) prediction results. Top: the actual (blue) and predictions (orange) for the differenced stationary time series. Bottom: after reversing the differencing.

The partial autocorrelation plot provided the order for the autoregressive model. The order is the lag after which the partial autocorrelation coefficients are not significantly different than zero. Using the plot (Figure 11), the order $p = 4$. The actual and predictions for both the differenced $AR(4)$ and the actual and predictions after reversing the differencing transformation are

shown in Figure 13. After reversing the differencing transformation, the model produced prediction evaluation metrics MSE 2147.96, MAE 24.42, and RMSE 46.35.

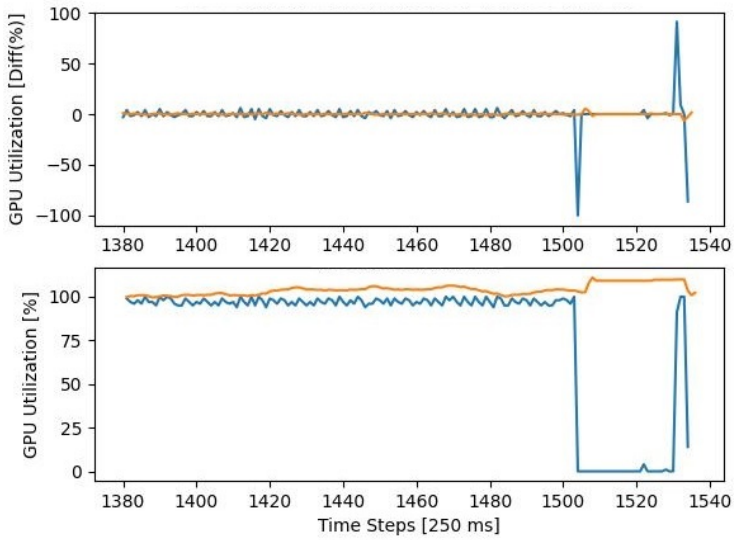

Figure 13. GPU utilization AR(4) prediction results. Top: the actual (blue) and predictions (orange) for the differenced stationary time series. Bottom: after reversing the differencing.

The ARMA model was created using the p and q orders found above, as described in Section 3.1.1. The ARMA model results are shown in Figure 14. The prediction evaluation results are MSE 4115.74, MAE 29.53, and an RMSE of 64.15. The AR model resulted in the lowest MSE, MAE and RMSE scores. This suggests that predictions depend linearly on the previous time steps. The abrupt change from the relatively flat GPU utilization time steps in the test data set explains why the moving average model is not as competitive. Since the ARMA model takes into account the moving average, it also is not as competitive as the AR model. According to the prediction evaluation results for the statistical non-seasonal results, the best model was $AR(4)$. The data at time step t can be modeled as

$$X_t = \varepsilon_t + -0.2233\varepsilon_{t-1} - 0.2014\varepsilon_{t-2} - 0.0504\varepsilon_{t-3} - 0.0395\varepsilon_{t-4}. \tag{8}$$

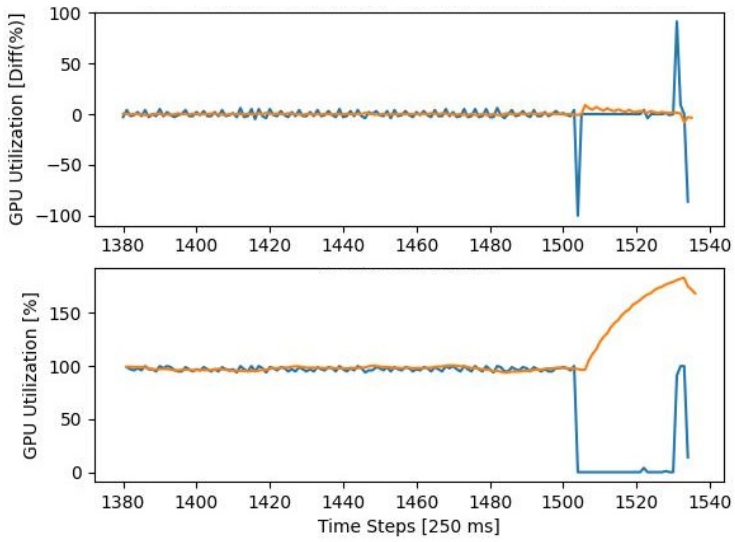

Figure 14. GPU utilization ARMA(4, 2) prediction results. Top: the actual (blue) and predictions (orange) for the differenced stationary time series. Bottom: after reversing the differencing.

4.1.2 Statistical Model: Power Draw

The power draw trace was not stationary according to the ADF test. The test statistic was not a large negative number and was not less than the critical value at a 1% significance level. The power draw trace was stationary according to the KPSS test. To make the power draw trace stationary, apply a detrending, the results shown in Figure 15. The ADF and KPSS test results from the detrended power draw trace verified that the detrending did produce a stationary series. The ADF test produced a large negative test statistic of -12.96, which was less than the critical value at a 1% significance level; and the p-value was zero. The KPSS test results from the detrended power draw trace produced a test statistic of 0.291, which was less than the critical value (0.347). It also produced a p-value of 0.1, which was greater than 0.01.

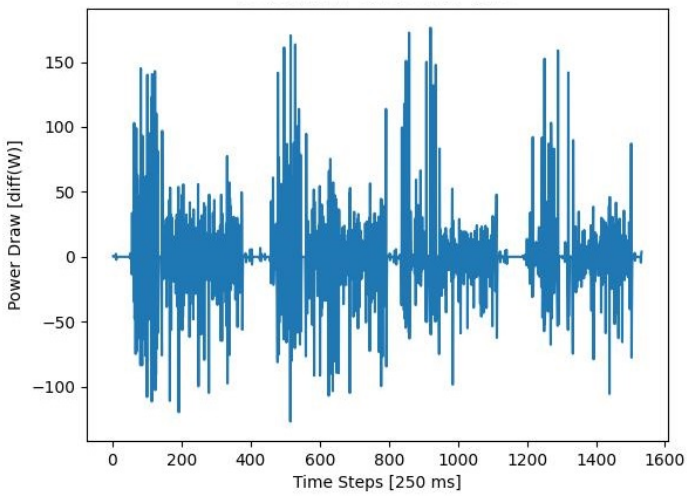

Figure 15. Power draw stationary trace created from applying a detrending transformation.

When plotting the ACF plot, the significant coefficients become non-significant after a lag of one. This means that the stationary series is not a random walk. Therefore, the detrended power draw series is stationary, see Figure 15 and Figure 16. A moving average prediction model can be applied with the lag $q = 1$ after which the lags become non-significant. Figure 17 displays the actual and predictions for both the reversing of the detrended stationary time series and for the original trace. The reversing of the detrending added the trend back to the prediction values. The prediction evaluation performance metrics resulted in an MSE of 1267.60, MSE of 21.04, and RMSE of 35.60.

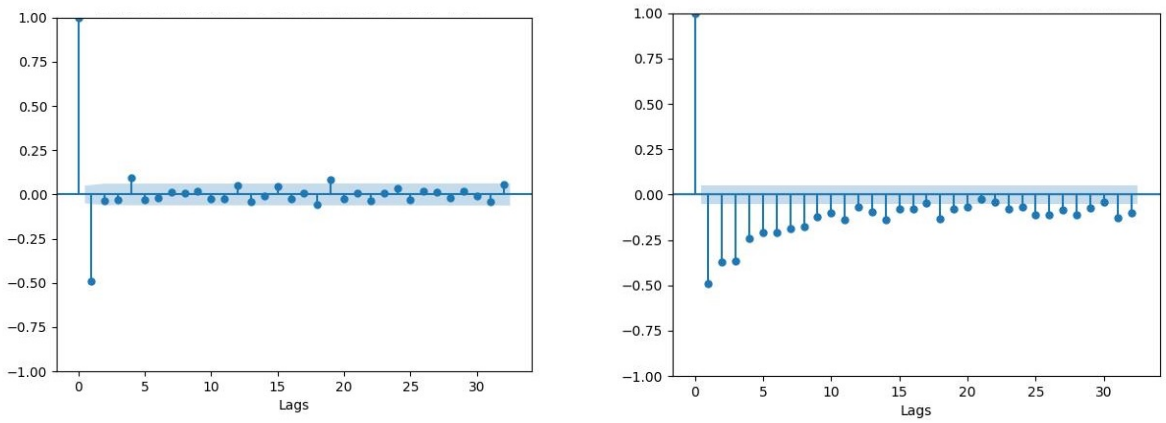

Figure 16. Left: an ACF plot for the detrended power draw trace. Lags are significant until lag 1. At lag 2, it falls below the shaded portion into the non-significance zone. Right: a PACF plot for the detrended power draw trace. Lags are significant until lag 16. At lag 17, it falls below the shaded portion into the non-significance zone.

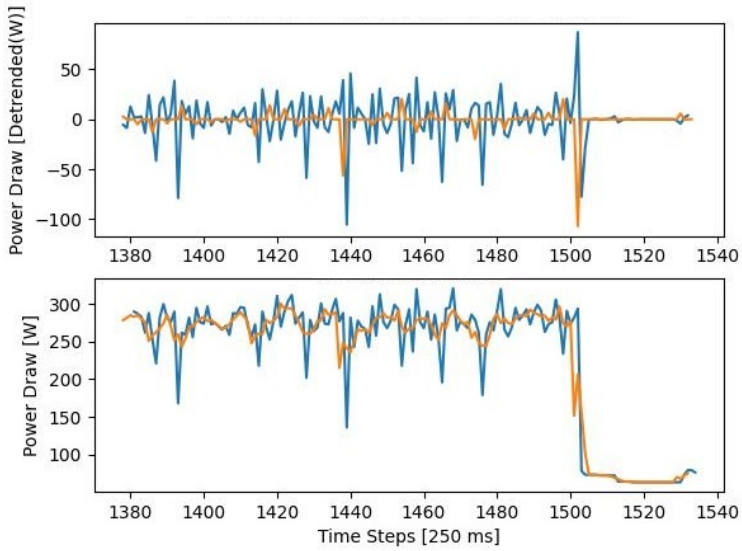

Figure 17. Power draw MA(1) prediction results. Top: actual (blue) and predictions (orange) for the detrended stationary time series. Bottom: after adding the trend back.

The next prediction model to try as shown in Section 3.1.1 is a moving average (MA) model. The model is created by viewing the partial autocorrelation plot, see Figure 16. A moving average model is created using the lag $p = 16$. Figure 18 shows the predictions and the actual values for the test data set both in the detrended and the reverse detrended form. This model resulted

in prediction evaluation metrics of MSE 1263.52, MAE 21.28, and RMSE of 35.54. The MSE and RMSE scores were lower, but the MAE was larger when comparing the AR and MA models.

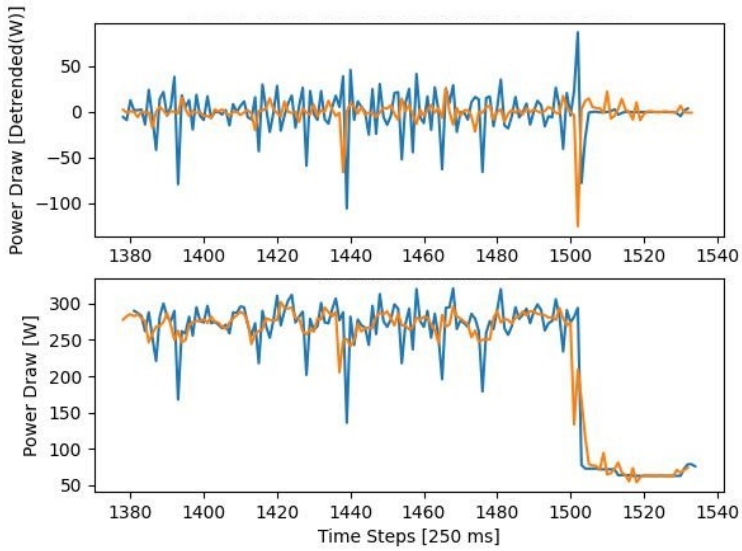

Figure 18. Power draw AR(16) prediction results. Top: actual (blue) and predictions (orange) for the detrended stationary time series. Bottom: after adding the trend back.

The ARMA model was created using the p and q orders found above, as described in Section 3.1.1. The ARMA model results are shown in Figure 19. The prediction evaluation results are MSE 1266.3, MAE 21.03, and an RMSE of 35.59. This ARMA model produced the lowest MAE score of the baseline models.

The ARMA results demonstrate that including the MA component, caused the MAE score to decrease by 0.25, but increased the MSE score by 2.77 and the RMSE score by 0.04. Therefore, in terms of the evaluation metrics collected, the $AR(16)$ outperforms the $MA(1)$ and $ARMA(16,1)$. This model can be written as

$$X_t = \varepsilon_t - 1.1388\varepsilon_{t-1} - 1.2874\varepsilon_{t-2} - 1.3754\varepsilon_{t-3} - 1.3196\varepsilon_{t-4} - 1.2873\varepsilon_{t-5} - 1.2215\varepsilon_{t-6}$$
$$- 1.1101\varepsilon_{t-7} - 0.9880\varepsilon_{t-8} - 0.8441\varepsilon_{t-9} - 0.7377\varepsilon_{t-10} - 0.6382\varepsilon_{t-11} - 0.4994\varepsilon_{t-12}$$
$$- 0.4058\varepsilon_{t-13} - 2.888\varepsilon_{t-14} - 0.1438\varepsilon_{t-15} - 0.0674\varepsilon_{t-16}. \quad (9)$$

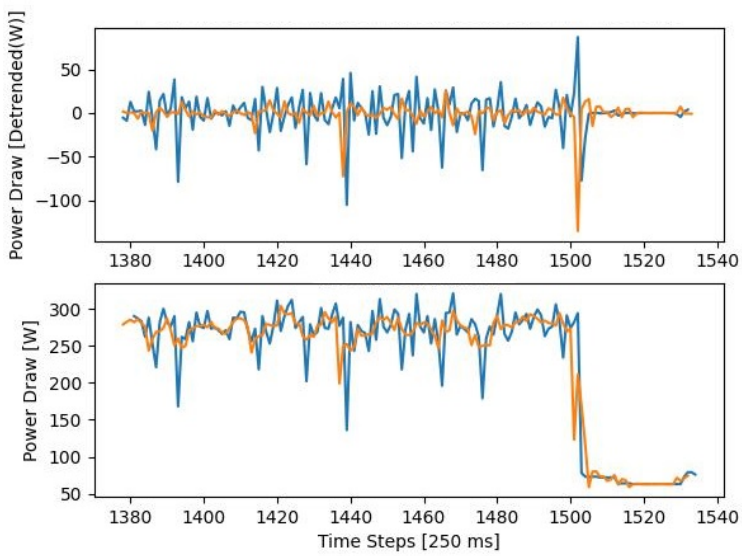

Figure 19. Power draw ARMA(16, 1) prediction results. Top: actual (blue) and predictions (orange) for the detrended stationary time series. Bottom: after adding the trend back.

The moving average models performed better on the power draw trace because though there were abrupt changes in the trace, it was not as abrupt.

4.1.3 Auto-ARIMA Results

A library that contains Auto-ARIMA [33] as explained in Section 3.1.2, allows the user to define the search space for the algorithm. Careful consideration must be taken to accommodate the computational memory limitations.

Two auto-ARIMA search spaces were explored for both the GPU utilization and power draw traces. One that assumed the time series was not seasonal and one that assumed that it was. For the GPU utilization search spaces, the input was the differenced series, and the prediction output reversed the differencing. Likewise, the input for the power draw was the detrended trace and the prediction output was then transformed to add the trend back. These inputs were provided to auto-ARIMA. Using the specified search method and search space, auto-ARIMA finds the model with the lowest AIC score. Results of the model used the rolling prediction on a horizon as explained in Section 3.1.2.

The first search space set *Seasonal = True*, meaning that the auto-ARIMA search iterated over possible non-seasonal and seasonal order values. The period and window size was set to four. The *max_order = 20*, where *max_order* limits the models so that $p + q + P + Q \leq max_order$. The algorithm searches the orders using a step-wise algorithm [33].

The results of seasonal auto-ARIMA on the stationary GPU utilization trace produced the model *SARIMA(3, 0, 1)(0, 0, 0)[4]*, resulting in a MSE 3680.34, MAE 29.81, and RMSE of 60.67. The test results are displayed in Figure 20. In terms of the evaluation metrics MSE, MAE, and RMSE, the *AR(4)* model outperformed this model. The results of the seasonal auto-ARIMA power draw trace produced the model *SARIMA(3, 0, 0)(7,0,0)[4]* (Figure 21), resulting in a MSE of 1451.73, MSE 22.84, and RMSE of 38.10. Similar to the results from the stationary GPU utilization trace, the *AR(16)* model outperformed this one in terms of MSE, MAE, and RMSE values. These results from both models demonstrate that minimizing the AIC does not equate to finding a model whose predictions produce low MSE, MAE, and RMSE values.

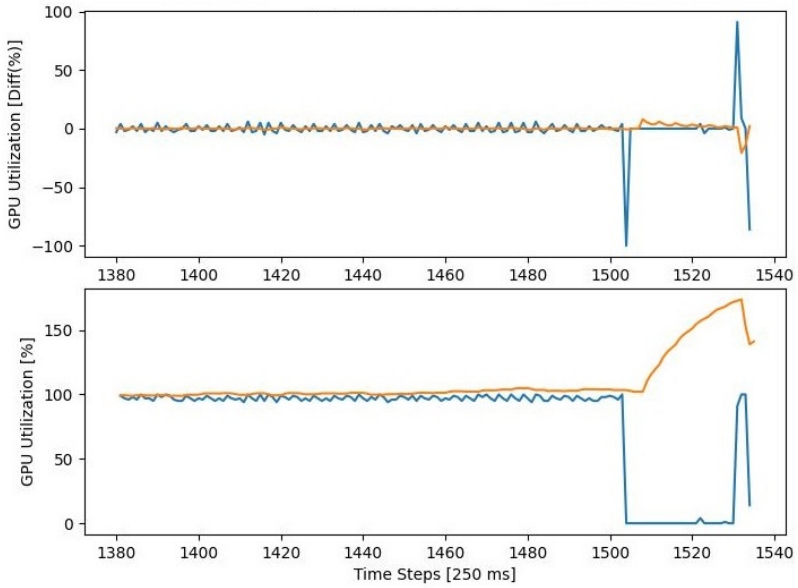

Figure 20. GPU utilization Auto-ARIMA prediction results. Top: actual and predictions for the differenced stationary time series. Bottom: after reversing the transformation.

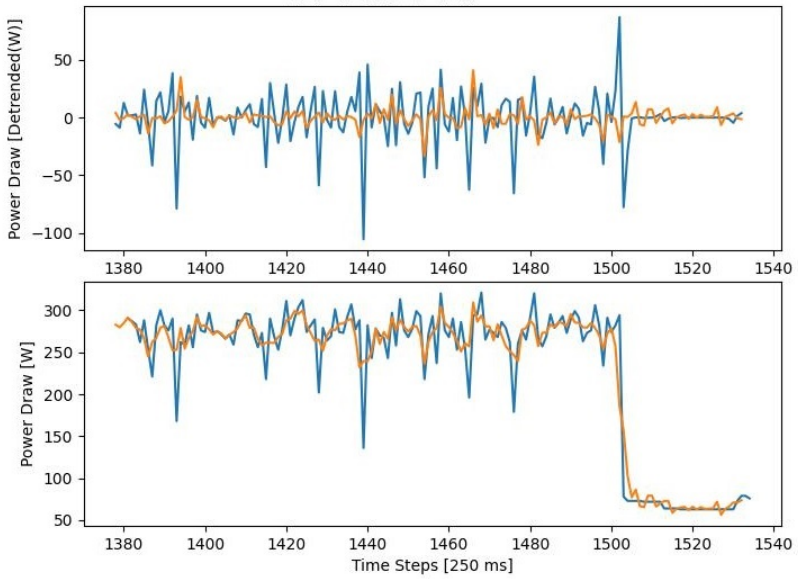

Figure 21. Power draw Auto-ARIMA prediction results. Top: actual and predictions for the detrended stationary time series. Bottom: after reversing the transformation.

The second auto-ARIMA run assumed that the trace was non-seasonal. Besides *Seasonal = FALSE*, the search method was the only difference. The search did not implement a stepwise search but used a grid search method. A grid search method used the "brute force" method where it increased the orders p and q, such that $p+q \leq 20$. Auto-ARIMA for the stationary GPU utilization trace (see Figure 22) found the optimal model in terms of AIC was $ARIMA(3,0,16)$. The prediction evaluation metrics were MSE 3689.2, MAE 29.99, and RMSE 60.74. This model performed worse than the $SARIMA(3,0,1)(0,0,0)[4]$, and thus $AR(4)$, in terms of the prediction evaluation metrics MSE, MAE, and RMSE. For the non-seasonal auto-ARIMA results on the stationary power draw trace, the model with the lowest AIC was $ARIMA(20,0,0)$, see Figure 23. The model produced prediction evaluation metrics MSE 1483.35, MAE 23.11, and RMSE 38.51. This model performed worse than the $SARIMA(3,0,0)(7,0,0)[4]$, and thus $AR(16)$, in terms of MSE, MAE, and RMSE.

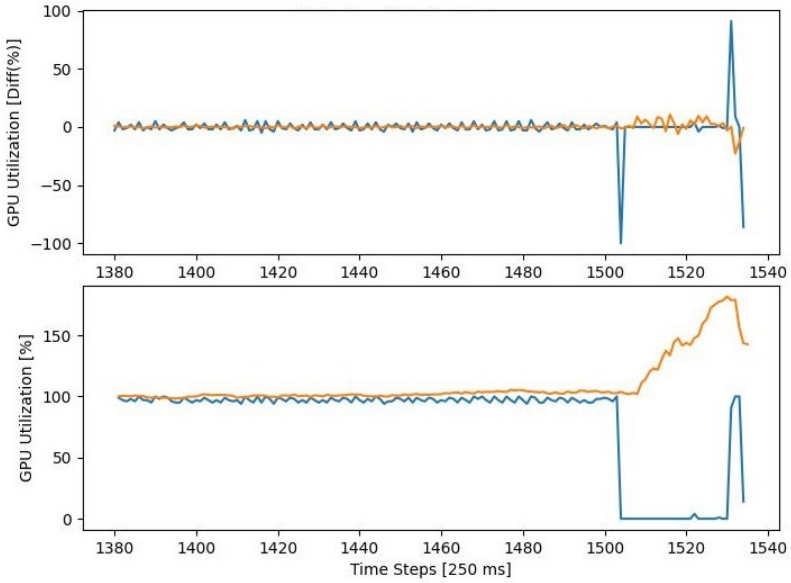

Figure 22. GPU utilization non-seasonal Auto-ARIMA prediction results. Top: actual and predictions for the differenced stationary time series. Bottom: after reversing the transformation.

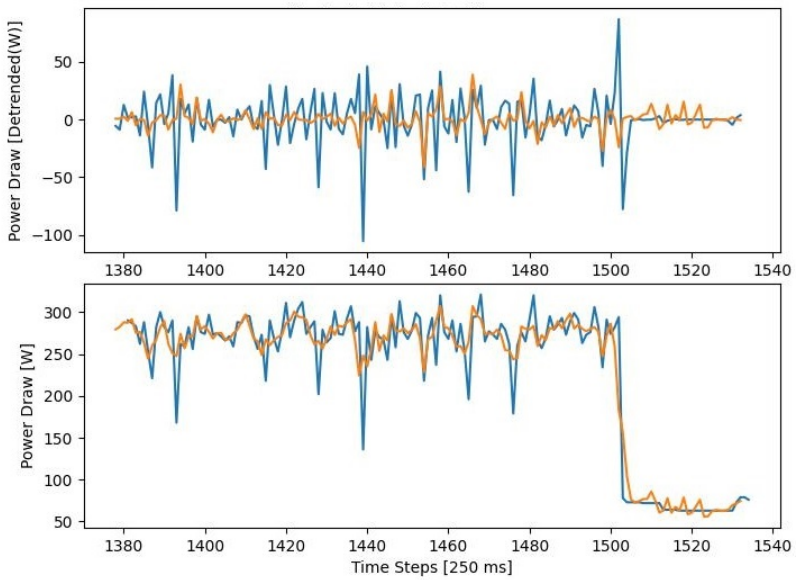

Figure 23. Power draw non-seasonal Auto-ARIMA prediction results. Top: actual and predictions for the detrended stationary time series. Bottom: after reversing the transformation.

The results of the auto-ARIMA verified that the statistical model with the lowest AIC score does not equate to low MSE, MAE, and RMSE scores. For both the stationary GPU utilization and power draw traces, the selected seasonal models outperformed the selected non-seasonal models in terms of MSE, MAE, and RMSE.

4.2 RECURRENT NEURAL NETWORK RESULTS

Analysis of the experimental results for the Recurrent Neural Networks (RNN), the SimpleRNN model and LSTM model (explained in Sections 3.3.1 and 3.3.2), focuses on the prediction evaluation metrics, MSE, MAE, and RMSE metrics. These metrics were recorded on the train, validation, and test data sets. Experimental results were taken for both univariate and multivariate input data sets. The input trace was scaled using the MinMaxScaler [26]. The data was transformed back to its original form before evaluation metrics were recorded. For the univariate results, the input only consisted of either the power draw or GPU utilization trace. For multivariate, both GPU utilization and power draw trace data were given.

To evaluate the RNN models properly, this section is organized as follows. Section 4.2.1 presents the experimental results and analysis of the univariate RNN (SimpleRNN and LSTM) for the GPU utilization trace. Section 4.2.1 does the same, but for the power draw trace. Then, multivariate RNN with the power draw trace is given as additional contextual information to predict GPU utilization data in Section 4.2.2. Then, multivariate RNN additional contextual information is GPU utilization data used to predict power draw in Section 4.2.2.

4.2.1 Univariate Recurrent Neural Network Results

The univariate trace input was provided with overlapping rolling windows with 24, 48, 65, 100, 160, and 185 past values (used to predict the next value). The number of past values is also referred to as "window length," or "look back" length. It is the number of recent time steps provided to the model as input (x_t) to make predictions (y_t). The overlapping rolling window method partitions a time series into multiple fixed-length segments. These segments are overlapping, meaning that the data from one window intersects with another. This overlapping rolling window method was applied to the train, validation, and test data sets, created using a 60-20-20 split. The trace was scaled using the MinMaxScaler [26]. Before collecting the performance evaluation metrics, the scaling was reversed. Metrics were collected on the train, validation, and test portions. The test MSE, MAE, and RMSE metrics provide insight into how the model predicts data not revealed to the model, and thus will be the focus for the analysis. The metrics for the training and validation sets provide more insight into the model's ability to predict.

Increasing the number of layers and increasing the number of units per layer increases the complexity of the model, and the capacity of the model to learn the mappings between inputs and outputs. There is a limit to increasing the number of layers and units per layer, where the model starts to over-fit, meaning that the model is not able to generalize well on unseen data. When increasing the number of layers and units per layer, consider the impacts on the runtime when compared with the amount of accuracy gains [56]. The optimizer selection affects how the weights are adjusted, which affects the rate of convergence and accuracy.

Univariate GPU Utilization RNN Model Results

For the univariate GPU Utilization RNN models, SimpleRNN and LSTM models were constructed using the same hyperparameters. The hyperparameters varied in the number of layers, the number of hidden units (per layer), the optimizer type, and the number of past values. The number of layers was either one, two, or three. The number of hidden units for a layer was either 32, 64, 84, or 128. The optimizer type was either RMSprop or Adam. The optimal model with its hyperparameter set, RNN type of either SimpleRNN or LSTM, window length (number of past values), optimizer type, and the layer units provided as a list, in terms of the given metric name and metric value is listed in Table 1. The W1.0 and W2.5 metrics state the percent of the predictions that fall within a relative difference of 1% and 2.5%, respectively to the actual.

Table 1. Univariate GPU utilization RNN model prediction evaluation metric results. The hyperparameter set that produced the optimal metrics is provided, along with the metric value. The RNN type refers to the RNN layer, either SimpleRNN or LSTM. The number of past values was 24, 48, 65, 100, 160, or 185. The optimizer was either Adam or RMSprop, and the number of layers as well as the number of hidden units per layer varied and are represented as a list.

Metric Name	RNN Type, Num. Past Values, Optimizer, Units	Metric Value
1. Train MSE	SimpleRNN, w = 185, Adam, [64, 32]	89.582
2. Train MAE	SimpleRNN, w = 185, Adam, [64, 32]	4.224
3. Train RMSE	SimpleRNN, w = 185, Adam, [64, 32]	9.465
4. Train Within 2.5%	LSTM, w = 160, Adam, [128, 84, 64]	58.870
5. Train Within 1.0%	SimpleRNN, w = 160, RMSprop, [200, 128, 64]	27.858
6. Val MSE	LSTM, w = 48, Adam, [64, 32]	125.844
7. Val MAE	SimpleRNN, w = 24, RMSprop, [200, 128, 64]	4.849
8. Val RMSE	LSTM, w = 48, Adam, [64, 32]	11.218
9. Val Within 2.5%	SimpleRNN, w = 24, RMSprop, [200, 128, 64]	46.996
10. Val Within 1.0%	SimpleRNN, w = 24, Adam, [200, 128, 64]	22.261
11. Test MSE	SimpleRNN, w = 160, RMSprop, [128, 84, 64]	143.311
12. Test MAE	**SimpleRNN, w = 100, RMSprop, [200, 128, 64]**	**3.722**
13. Test RMSE	SimpleRNN, w = 160, RMSprop, [128, 84, 64]	11.971
14. Test Within 2.5%	SimpleRNN, w = 160, Adam, [200, 128, 64]	66.667
15. Test Within 1.0%	LSTM, w = 100, Adam, [64]	32.800

Figure 24 displays the prediction results for the SimpleRNN model that produced the lowest MSE and RMSE metric values. The hyperparameter set is described in Table 1, lines 11 and 13. Figure 25 displays the prediction results for the SimpleRNN model that produced the lowest MAE metric value. The hyperparameter set is described in Table 1, line 12. Figure 26 displays the prediction results for the model that produced the lowest W2.5. The hyperparameter set is described in Table 1, line 14. Figure 27 displays the prediction results for the model that produced the lowest W1.0. The hyperparameter set is described in Table 1, line 15.

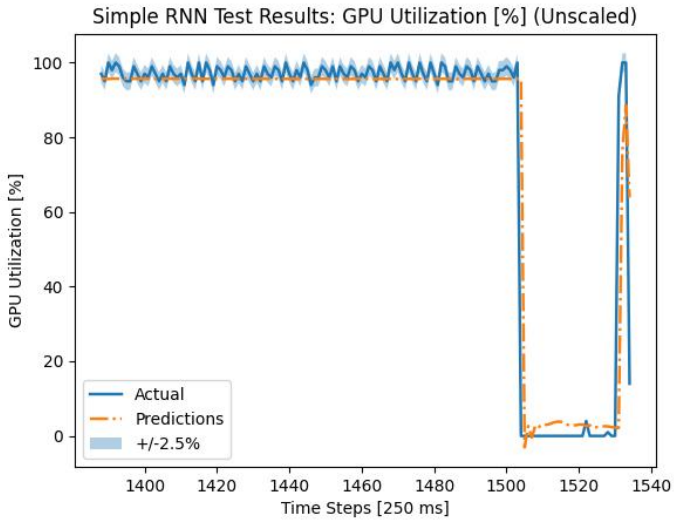

Figure 24. GPU utilization Univariate RNN prediction results that produced the lowest test MSE and RMSE values. See Table 1, lines 11 and 13, for the hyperparameter set information.

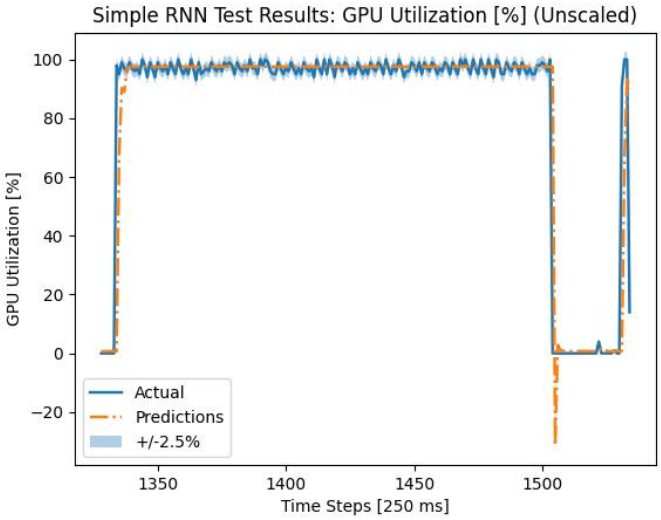

Figure 25. GPU utilization Univariate RNN prediction results that produced the lowest test MAE value. See Table 1, line 12, for the hyperparameter set information.

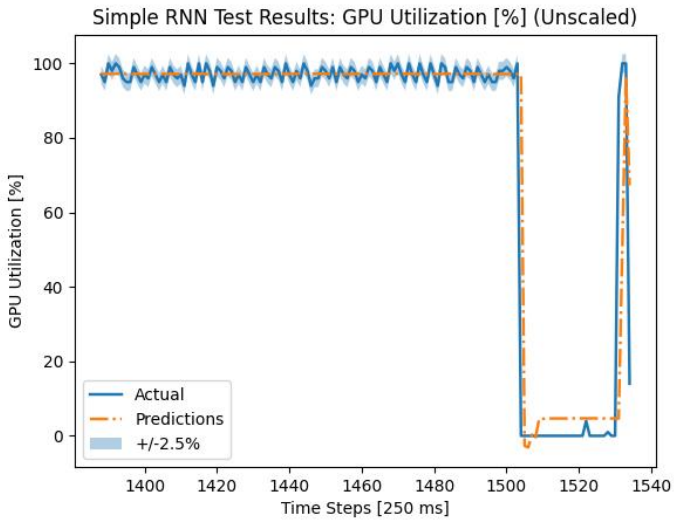

Figure 26. GPU utilization Univariate RNN prediction results that produced the lowest test Within 2.5% value. See Table 1, line 14, for the hyperparameter set information.

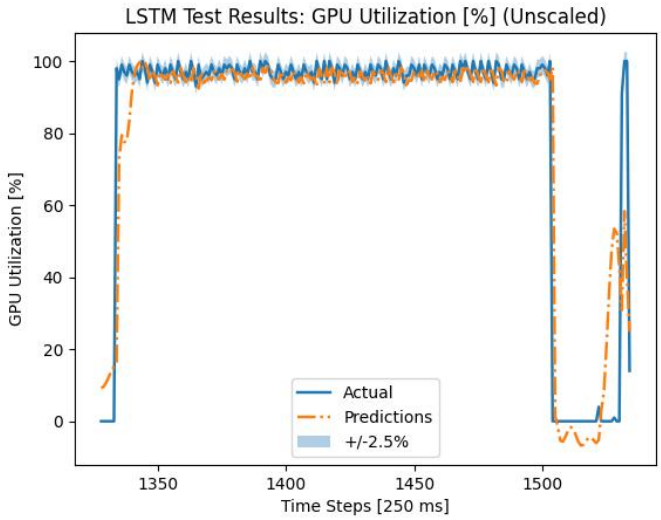

Figure 27. The GPU utilization Univariate RNN model prediction results that produced the lowest Within 1.0% value. See Table 1, line 15, for the hyperparameter set information.

For the train data set, a SimpleRNN model with two layers, with 64 and 32 hidden layers, performed well in all of the metric categories except for the within 2.5% and within 1.0% category. Also, for the train evaluation metrics, the models that performed well used a larger number of past values, 160. Within the validation data set, the model's three layers, with 200, 128, and 64 units per layer, produced the lowest MAE and Within 2.5% and Within 1.0% metric values. For the test data set, a SimpleRNN model with three layers using RMSprop produced the lowest MSE, MAE, and RMSE. It also produced the lowest Within 2.5%. For all the metrics, the models that produced the optimal metrics utilized either 100 or 160 past values.

For the set of hyperparameters that produced the lowest test MSE scores, the second lowest model was the LSTM version of model listed in Table 1, line 11. The LSTM model, in terms of MSE, was better able to predict the high utilization portion. However, this gain was overshadowed by the much larger MSE increase due to the predictions in the low utilization portion. The MSE value penalized the larger errors more. The following two models that produced the lowest MSE were LSTM models with 100 past values.

For the set of hyperparameters that produced the lowest test MAE scores, the second lowest model was the model that produced the lowest MSE score, listed in Table 1, line 11. The model predictions in Figure 25 and Figure 24 show that the prior's predictions, for the most part, were closer to the actual values, except for the time step where the trace data suddenly changed from high to low. The prior model did not perform well according to the MSE metric because of the large difference at said point (since MSE strongly penalizes larger errors). In other words, the MAE metric does not penalize larger errors as much as the MSE metric. The other two following lowest test MAE models were also SimpleRNN models with 160 past values.

The prediction evaluation results from the test data set suggest that models with a longer number of past values produce lower metrics and in general, SimpleRNN models outperformed their LSTM equivalents.

Univariate Power Draw RNN Model Results

Univariate power draw RNN model applied the same hyperparameter sets to both SimpleRNN and LSTM. The other hyperparameters varied were the number of layers and units per layer, and the number of past values. The input trace was given in the overlapping rolling window method with 24, 48, 65, 100, 160, and 185 past values. The number of past values and the RNN layer configurations producing the most optimal metric values are provided in Table 2. As explained in Section 4.2.1, the metrics compare the predictions after the reversal of the scaling.

Table 2. Univariate power draw RNN model prediction evaluation metric results. The hyperparameter set that produced the optimal metrics is provided, along with the metric value. The RNN type refers to the RNN layer, either SimpleRNN or LSTM. The number of layers and the number of units per layer are represented as a list.

Metric Name	Power Draw RNN Type, Num. Past Values, Layer units	Metric Value
1. Train MSE	SimpleRNN, w = 48, [128, 64]	981.149
2. Train MAE	SimpleRNN, w = 65, [128, 64]	21.044
3. Train RMSE	SimpleRNN, w = 48, [128, 64]	31.323
4. Train Within 2.5%	SimpleRNN, w = 100, [64, 32]	23.143
5. Train Within 1.0%	SimpleRNN, w = 100, [128, 84, 64]	11.084
6. Val. MSE	LSTM, w = 185, [64, 32, 32]	499.859
7. Val. MAE	SimpleRNN, w = 185, [64, 32, 32]	8.791
8. Val. RMSE	LSTM, w = 185, [64, 32, 32]	22.358
9. Val. Within 2.5%	SimpleRNN, w = 185, [64, 32, 32]	46.721
10. Val. Within 1.0%	SimpleRNN, w = 100, [128, 84, 64]	26.570
11. Test MSE	LSTM, w = 100, [200, 128, 64]	958.064
12. Test MAE	**SimpleRNN, w = 65, [128, 84, 64]**	**17.872**
13. Test RMSE	LSTM, w = 100, [200, 128, 64]	30.953
14. Test Within 2.5%	SimpleRNN, w = 160, [64, 32]	29.252
15. Test Within 1.0%	SimpleRNN, w = 65, [128, 84, 64]	13.636

The prediction results for the hyperparameter set, defined in Table 2 lines 11 and 13, that produced the lowest test MSE and RMSE metrics is displayed in Figure 28. The prediction results for the hyperparameter set, defined in Table 2 line 12, that produced the lowest test MAE metric is displayed in Figure 29. The prediction results for the hyperparameter set, defined in Table 2 line 14, that produced the highest W2.5 metric is displayed in Figure 30. The prediction results for the hyperparameter set, defined in Table 2 line 15, that produced the highest W1.0 metric is displayed in Figure 31.

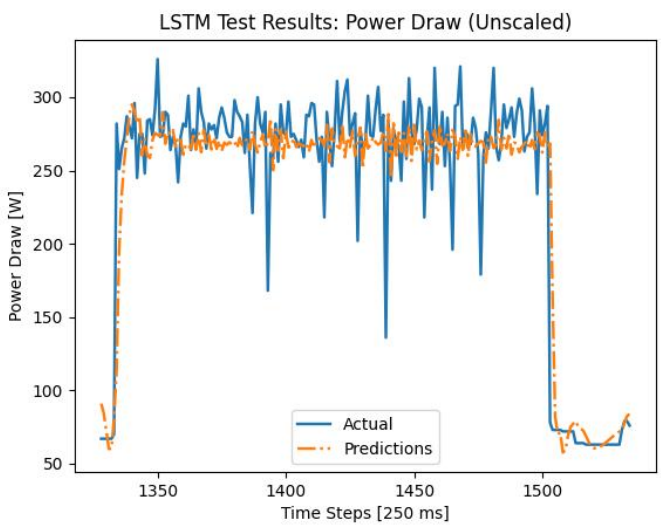

Figure 28. Univariate power draw RNN model prediction results that produced the lowest MSE and RMSE metrics. See Table 2, lines 11 and 13, for the hyperparameter set information.

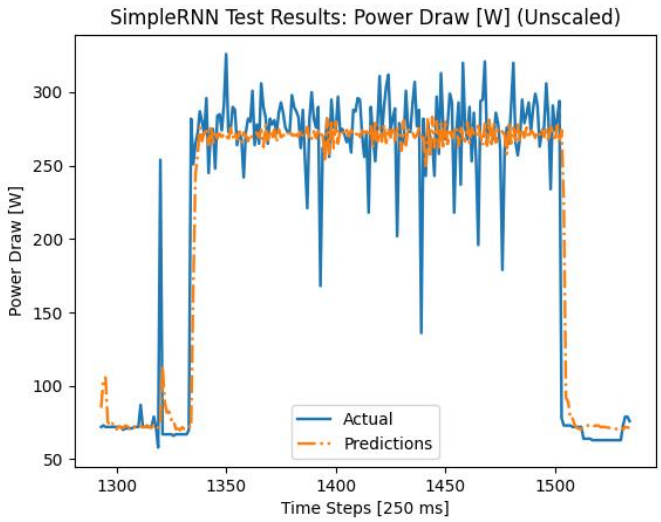

Figure 29. Univariate power draw RNN model prediction results that produced the lowest test MAE metric. See Table 2, line 12, for the hyperparameter set information.

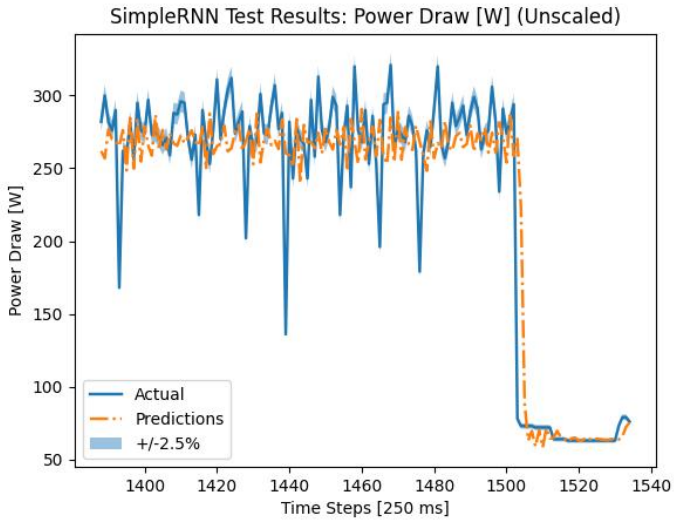

Figure 30. Univariate power draw RNN model prediction results that produced the lowest test W2.5 metric. See Table 2, line 14, for the hyperparameter set information.

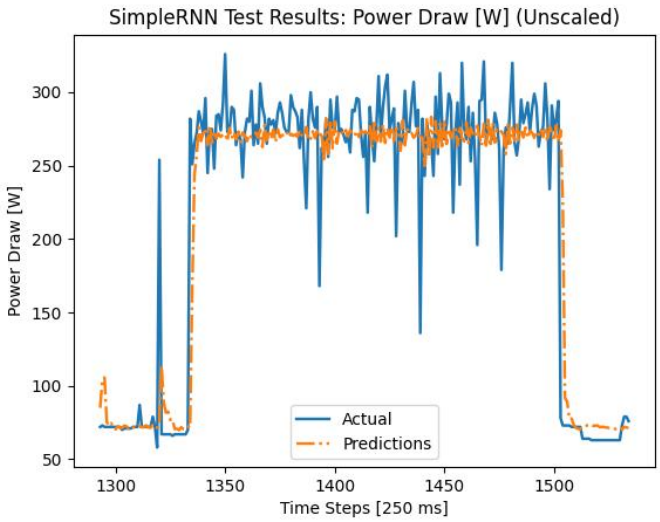

Figure 31. Univariate power draw RNN model prediction results that produced the lowest test W1.0 metric. See Table 2, line 15, for the hyperparameter set information.

Models with two SimpleRNN layers, 128 and 64 units, produced the lowest train MSE, MAE, and RMSE metrics, see Table 2 lines 1-3. The number of past values that produce the lowest train evaluation metrics is less than 65. The LSTM equivalent that produced the lowest MAE metric, see Table 2 line 7, produced the lowest validation MSE and RMSE metrics. The models that produce the lowest test MSE, MAE, and RMSE evaluation metrics utilized three layers.

The hyperparameter set that produced the second lowest test MSE metric was a SimpleRNN model with the same layer configuration (as the one described in Table 2 line 11), but a different number of past values. The next two models that produce the lowest test MSE metrics utilize 64, 32, and 32 hidden units per layer, with the SimpleRNN model producing a slightly lower test MSE metric than its LSTM equivalent. The hyperparameter sets that produce the first, second, and fourth lowest test MSE scores use a SimpleRNN model that learns using three layers. Curiously, the model producing the third lowest test MAE metric was built using only one layer.

The experimental results demonstrated that the LSTM models produced lower MSE and RMSE scores on new data (as shown by the validation and test results in Table 2 lines 6, 8, 11, and 13). Interestingly, the LSTM model did not produce lower metrics within the training set.

4.2.2 Multivariate Recurrent Neural Network Results

Multivariate RNN's use additional variables to assist in the prediction of a singular variable output. Multivariate models use both the previous data of the variable to predict and any other variables provided. It assumes that the other variable's past data points assist in accurately predicting the output. In other words, it assumes there is a pattern that exists between the different variables. The multivariate model trained on the scaled trace data, but the predictions were transformed back into the original trace data format before the metrics and prediction visualizations were created. The hyperparameters that varied between models included the number of layers (one, two, or three), and the number of hidden units, either 200, 128, 84, 64, or 32. It also included the optimizer type, either RMSprop or Adam. The same set of hyperparameters was provided to both the SimpleRNN and LSTM models.

Multivariate RNN Predicting GPU Utilization Results

Both the trace data for the power draw and GPU utilization data were provided in a multivariate manner. The power draw trace data was used to assist in predicting the GPU utilization data. The optimal model, with its hyperparameter set, in terms of a given metric, are provided in 3. Figure 32 displays the prediction results for the model that produced the lowest MSE and RMSE prediction evaluation results on the test data set. The hyperparameter set is described in Table 3, lines 11 and 13. Figure 33 displays the prediction results for the model that produced the lowest MAE. The hyperparameter set is described in Table 3, line 12. Figure 34 displays the prediction results for the model that produced the highest percentage of data points contained in the test data set that fell within 2.5% and 1.0% of the actual value. The hyperparameter set is described in Table 3, lines 14 and 15.

Table 3. Multivariate GPU utilization RNN prediction evaluation metrics. The hyperparameter set that produced the optimal metrics is provided, along with the metric value. The RNN type refers to either SimpleRNN or LSTM layers. The number of past values, the optimizer was Adam or RMSprop, and the number of layers, as well as the number of units per layer varied and are represented as a list.

Metric Name	RNN Type, Num. Past Values, Optimizer, Layer Units	Metric Value
1. Train MSE	Mult. LSTM, w = 24, RMSprop, [200, 128, 64]	49.902
2. Train MAE	Mult. LSTM, w = 24, RMSprop, [200, 128, 64]	3.812
3. Train RMSE	Mult. LSTM, w = 24, RMSprop, [200, 128, 64]	7.064
4. Train Within 2.5%	Mult. Simple RNN, w = 160, Adam, [128, 84, 64]	62.500
5. Train Within 1.0%	Mult. LSTM, w = 24, RMSprop, [200, 128, 64]	30.992
6. Val MSE	Mult. LSTM, w = 24, Adam, [64, 64]	124.028
7. Val MAE	Mult. Simple RNN, w = 24, Adam, [64, 32, 32]	4.387
8. Val RMSE	Mult. LSTM, w = 24, Adam, [64, 64]	11.137
9. Val Within 2.5%	Mult. Simple RNN, w = 24, RMSprop, [200, 128, 64]	47.703
10. Val Within 1.0%	Mult. LSTM, w = 24, RMSprop, [200, 128, 64]	20.141
11. Test MSE	Mult. LSTM, w = 160, RMSprop, [128, 84, 64]	138.334
12. Test MAE	**Mult. Simple RNN, w = 160, RMSprop, [128, 84, 64]**	**3.738**
13. Test RMSE	Mult. LSTM, w = 160, RMSprop, [128, 84, 64]	11.762
14. Test Within 2.5%	Mult. Simple RNN, w = 100, RMSprop, [200, 128, 64]	67.633
15. Test Within 1.0%	Mult. Simple RNN, w = 100, RMSprop, [200, 128, 64]	28.019

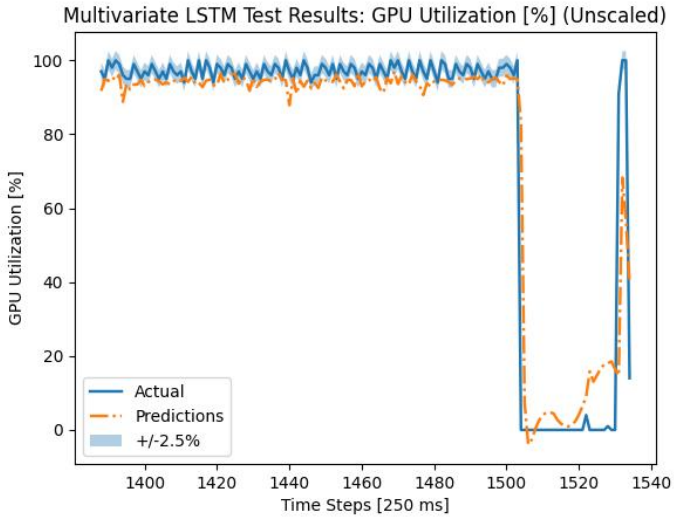

Figure 32. The GPU utilization Multivariate RNN model prediction results that produced the lowest test MSE and RMSE. See Table 3, lines 11 and 13 for the hyperparameter information.

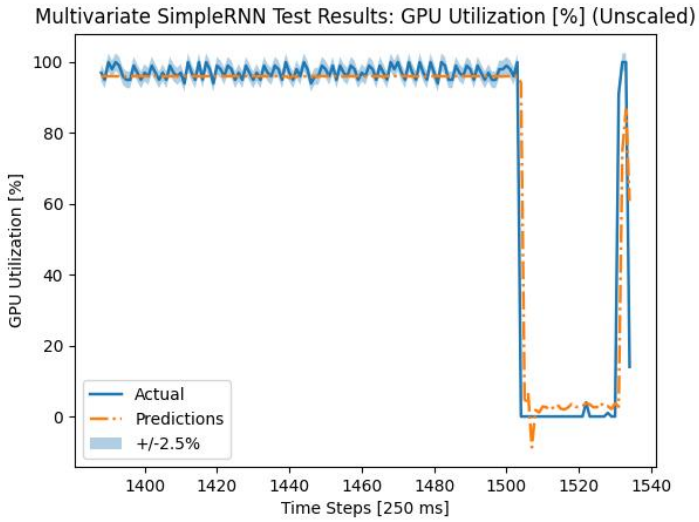

Figure 33. The GPU utilization Multivariate RNN model prediction results that produced the lowest test MAE. See Table 3, line 12, for the hyperparameter information.

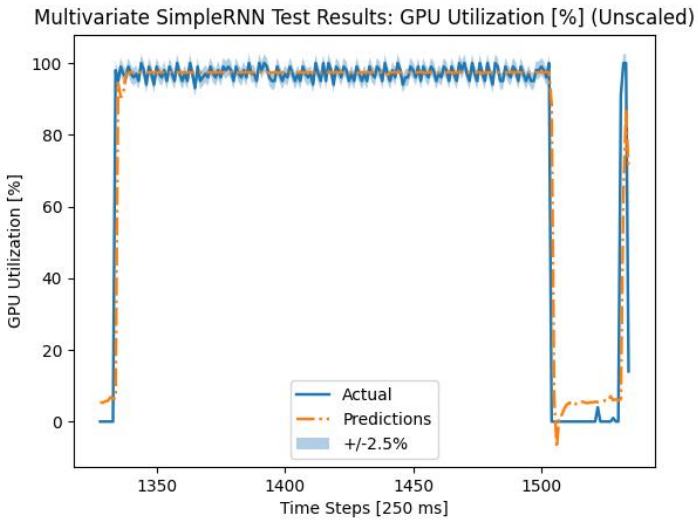

Figure 34. The GPU utilization Multivariate RNN model prediction results produced the highest percentage of predictions that were within a relative tolerance of 2.5% and 1.0%. See Table 3, lines 14 and 15, for the hyperparameter information.

Besides the W2.5 metric, in terms of the train prediction evaluation metrics, the most optimal model was a multivariate LSTM model with 24 past values, using an RMSprop optimizer, and 200, 128, and 64 units per layer. In terms of the validation prediction evaluation metrics, the most optimal models used 24 past values. For the validation of MSE, MAE, and RMSE, the hyperparameter set utilized an Adam optimizer with the first RNN layer containing 64 units. In terms of the test prediction evaluation metrics, the most optimal model contained three layers and used a window length of either 100 or 160, with an RMSprop optimizer. The hyperparameter set that produced the lowest test MAE was the same as the one that produced the lowest MSE and RMSE, except for the RNN layer type.

Empirical results for the hyperparameter sets that produced the lowest three test MSE demonstrated the following commonalities. The models built using the layers and optimizer hyperparameters listed in Table 3, lines 11 and 13, produced lowest test MSE values. The model producing the second lowest test MSE utilized SimpleRNN layers and the same number of past values as the first. It is also the model described in Table 3, line 12. The third lowest was pro-

duced by a model that utilized LSTM layers, but 185 past values.

Empirical results for the hyperparameter sets that produced the lowest three test MAE scores demonstrated the following commonalities. They were SimpleRNN models with three layers and 100+ past values.

Multivariate RNN Predicting Power Draw Results

This type of model provides to the RNN's both the trace data for the power draw and GPU utilization data. The GPU utilization data is given as additional information to the model. The hyperparameter set for the model that produced the most optimal metric value (lowest for MSE, MAE, and RMSE, but highest for the within percentage metrics) is provided in Table 4. The hyperparameter set varied in the type of RNN layer, SimpleRNN or LSTM. It also varied the number of past values, the optimizer Adam or RMSprop, and the layer configuration (the number of layers and the number of units per layer).

Figure 35 displays the prediction results for the model that produced the lowest MSE and RMSE prediction evaluation results on the test data set. The hyperparameter set is described in Table 4, lines 11 and 13. Figure 36 displays the prediction results for the model that produced the lowest MAE. The hyperparameter set is described in Table 4, line 12. Figure 37 displays the prediction results for the model that produced the highest percentage of data points contained in the test data set that fell within 2.5% of the actual value. The hyperparameter set is described in Table 4, line 13. Figure 38 displays the prediction results for the model that produced the highest percentage of data points contained in the test data set that fell within 1.0% of the actual value. The hyperparameter set is described in Table 4, line 15.

Table 4. Multivariate power draw RNN prediction evaluation metrics. The hyperparameter set that produced the optimal metrics are provided, along with the metric value. The RNN type refers to either SimpleRNN or LSTM layers. The number of past values is also listed. The optimizer was either Adam or RMSprop. The number of layers as well as the number of units per layer varied and are represented as a list.

Metric Name	RNN Type, Num. Past Values, Optimizer, Layer Units	Metric Value
1. Train MSE	LSTM: Len. 24, Adam, [128, 84, 64]	1344.056
2. Train MAE	SimpleRNN: Len. 160, Adam, [64, 32, 32]	23.336
3. Train RMSE	LSTM: Len. 24, Adam, [128, 84, 64]	36.661
4. Train Within 2.5%	LSTM: Len. 100, Adam, [128, 84, 64]	21.628
5. Train Within 1.0%	LSTM: Len. 160, RMSprop, [64]	10.280
6. Val. MSE	LSTM: Len. 185, Adam, [64, 32]	490.002
7. Val. MAE	SimpleRNN: Len. 185, Adam, [64, 32]	9.111
8. Val. RMSE	LSTM: Len. 185, Adam, [64, 32]	22.136
9. Val. Within 2.5%	SimpleRNN: Len. 100, RMSprop, [64]	41.546
10. Val. Within 1.0%	SimpleRNN: Len. 100, RMSprop, [64]	24.155
11. Test MSE	SimpleRNN: Len. 160, Adam, [200, 128, 64]	1078.538
12. Test MAE	**SimpleRNN: Len. 65, Adam, [128, 84, 64]**	**17.253**
13. Test RMSE	SimpleRNN: Len. 160, Adam, [200, 128, 64]	32.841
14. Test Within 2.5%	SimpleRNN: Len. 160, Adam, [64, 32, 32]	33.333
15. Test Within 1.0%	SimpleRNN: Len. 48, Adam, [128, 84, 64]	18.533

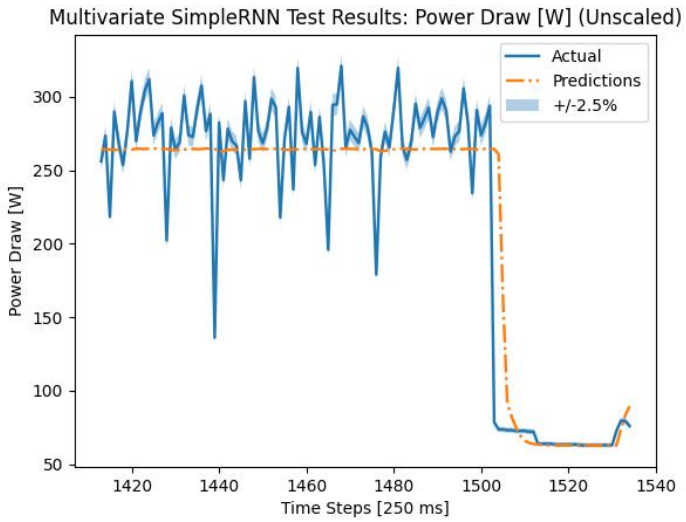

Figure 35. The power draw Multivariate RNN model prediction results that produced the lowest test MSE and RMSE. See Table 4, lines 11 and 13, for the hyperparameter information.

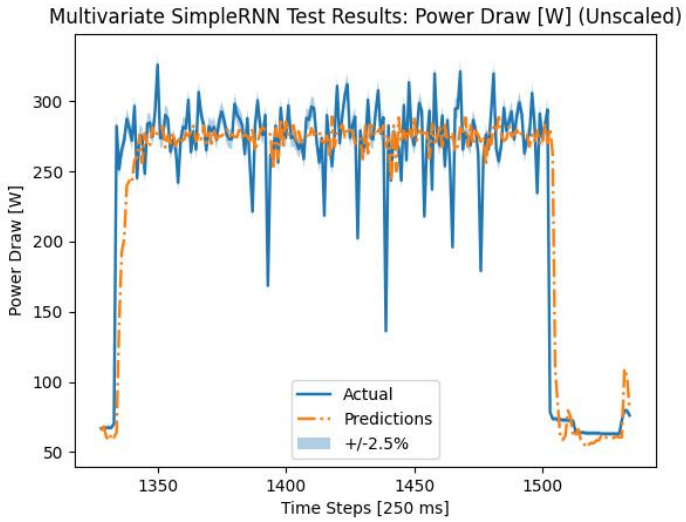

Figure 36. The power draw Multivariate RNN model prediction results that produced the lowest test MAE. See Table 4, line 12, for the hyperparameter information.

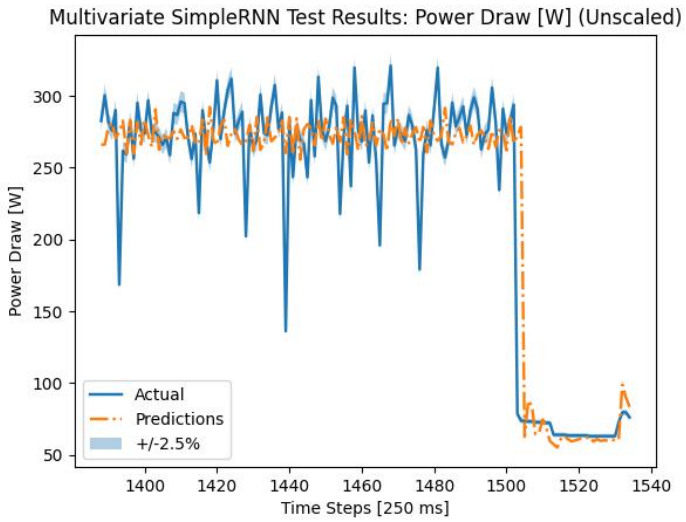

Figure 37. The power draw Multivariate RNN model prediction results that produced the lowest test W2.5. See Table 4, line 14, for the hyperparameter information.

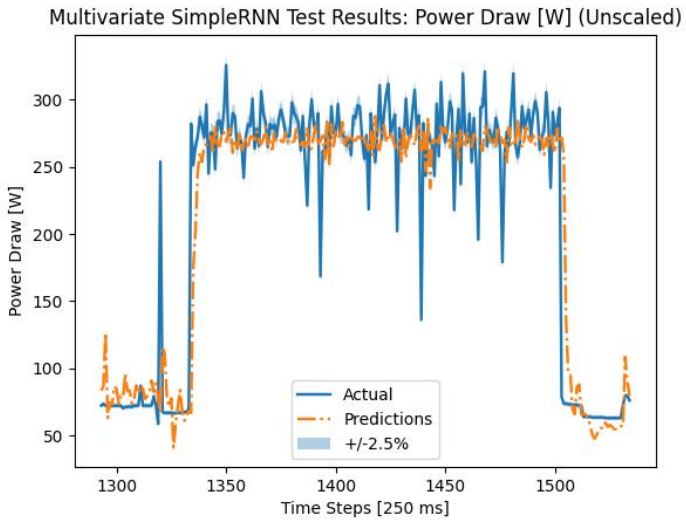

Figure 38. The power draw Multivariate RNN model prediction results that produced the lowest test W1.0. See Table 4, line 15, for the hyperparameter information.

The experimental results demonstrated that the optimal model, according to it's associated metric in Table 4, utilized an Adam optimizer, with the exception of lines 5, 9, and 10. The same table of results demonstrates that the LSTM models produced the lowest train metric re-

sults, with the exception of the model producing the lowest train MAE. In addition, with the exception of the model producing the lowest W1.0 metric, the models producing the lowest train metrics employed an Adam optimizer with three layers. The same model hyperparameters, except the RNN type of layer, produced the lowest validation MSE, MAE, and RMSE metrics (see Table 4). The exact same model produced the lowest validation W2.5 and W1.0 metrics. All of the models that produced the lowest validation metrics used 100+ past values. The models that produced the lowest test prediction evaluation metrics (see Table 4) used three SimpleRNN layers and an Adam optimizer.

Looking at the models that produced the lowest three test MSE metrics, all of them use an Adam optimizer with three layers. Except for the second lowest, the other models utilize a SimpleRNN layer type. The number of past values was either 100 or 160. Looking at the models that produced the lowest three test MAE metrics, all of them use an Adam optimizer with three SimpleRNN layers.

4.3 TRANSFORMER NEURAL NETWORK RESULTS

Analysis of the experimental results for the univariate Probabilistic TNN model, described in Section 3.4 focuses on the prediction evaluation metrics. The prediction evaluation metrics are the median predictions for each sequential trace in the “global” set of test data sets. The metrics MSE and MAE included in Table 6 are the median of those median metrics. The metrics were collected on the scaled traces. The blue region in the actual and prediction plots represents the uncertainty, defined as +/- one standard deviation from the prediction. To find the model that produced the optimal results, first, the mean metric values were examined shown in the Tables. The resulting model with the most optimal metric value was selected. Then, the time series trace that produced the optimal metric value is displayed in the resulting plots.

The forecast horizon length varied from 24, 48, 65, 160, 185, 200, 225, and 300. Recall from Section 3.4.1 that the forecasting horizon length is the number of time steps to predict. The prediction length is a function of the context length. Experimental models varied the number

of encoder and decoder layers from 4 to 6 for the GPU utilization models, and the power draw models hyperparameter search space was increased to include varying the number of time series traces provided to the model, either 9, 25, or 40. It also varied the type of distribution and the number of dimensions (*d_model*).

For the univariate Annotated TNN, the hyperparameter search for the GPU utilization was limited to varying the number of encoder and decoder layers and the amount of smoothing for the Label Smoother, either 0.025 or 0.1. Label smoothing is implemented to increase the uncertainty of the weights. For the power draw, it varied the amount of layers. As explained in Section 3.4.2, the inputs to the model are not scaled and are in integer form because there is a mapping of the data value and its positional information (similar to translation models). This model was constructed to distribute the work among three parallel GPUs. For each of the batches in the test data set, distributed among the parallel GPUs, the batch that resulted in the lowest MSE and MAE scores were plotted.

4.3.1 TNN Probabilistic GPU Utilization Results

The inputs to the model were 25-time series traces. The hyperparameters was the forecasting length, number of encoder and decoder layers, the dimensions (*d_model*), and distribution type. The forecasting lengths were 24, 48, 65, 160, 185, 200, 225, and 300. The number of layers was either 4 or 6. The distribution type was either student-t or normal, this was due to limitations set by the original code base [43]. It was observed that a forecasting length of less than 185 were not able to predict the sudden change in trace data from high to low, and vice-versa. Thus, they will not be included in the analysis. The lowest trace metrics from the model with the lowest mean metrics (over all of the traces) is included in Table 5 and its predictions and mean metrics per trace are displayed in Figure 39. For the given experimental results, the same model with a *d_model* of 32 (instead of 64), produced identical scaled metrics.

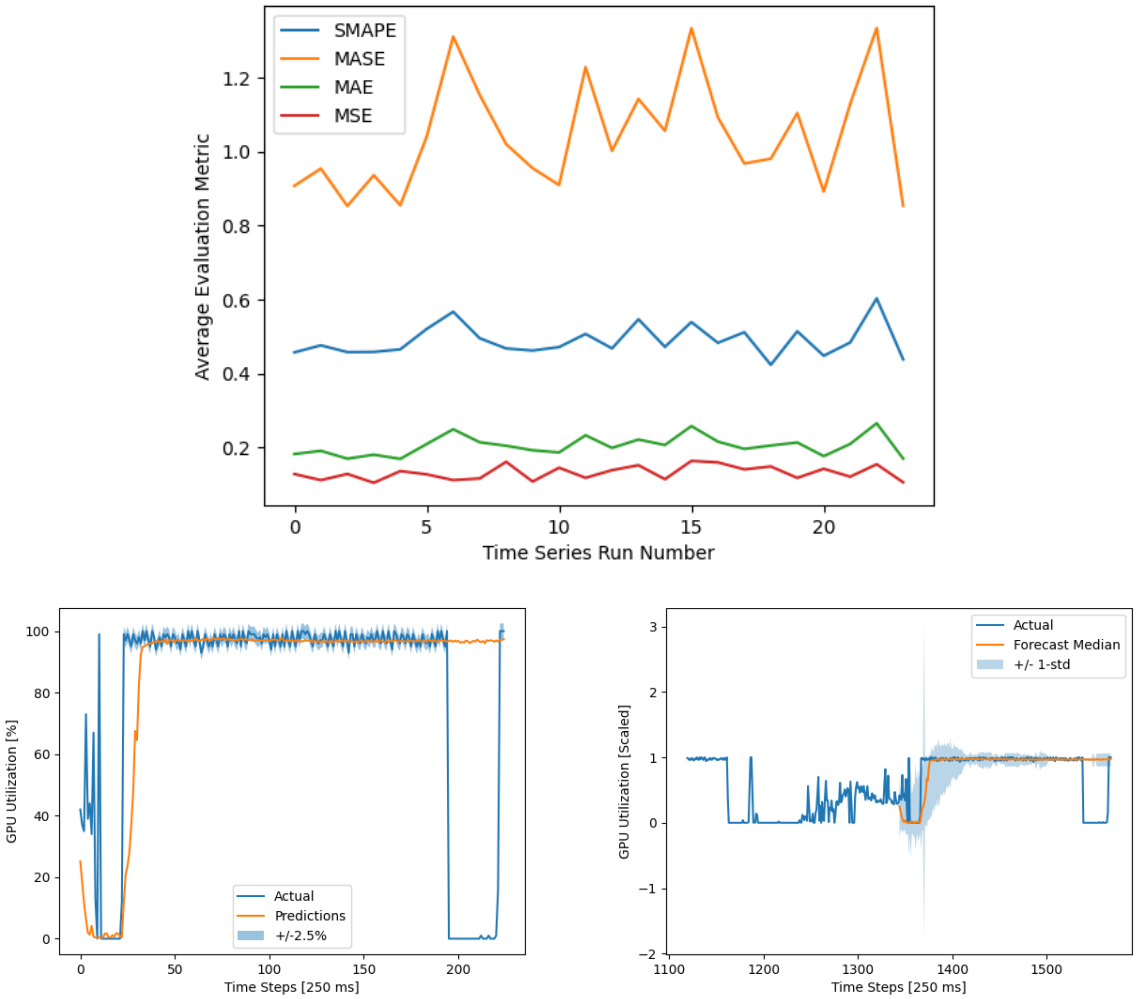

Figure 39. GPU utilization Probabilistic TNN prediction results and metrics per trace. Top: MAE and MSE metrics per trace for the model hyperparameters that produced the lowest MAE and MSE metrics. Bottom row: Prediction results for Trace 4, not scaled on the left and scaled on the right (with uncertainty +/- 1 standard deviation).

Table 5. GPU utilization Probabilistic TNN prediction results for the unscaled trace data that produced the lowest MSE and MAE. The model's hyperparameters are four encoder and decoder layers, student-t distribution, a *d_model* of 32, and 25 traces. The forecasting length was 225.

Trace Number	MSE	MAE
4	1358.290	16.889

4.3.2 TNN Probabilistic Power Draw Results

The hyperparameters was the forecasting length, number of traces, number layers, distribution type, and the dimensions (*d_model*). The forecasting lengths were 24, 48, 65, 160, 185, 200, 225, and 300. The number of traces was either 9, 25, or 40. The number of layers was either 2, 4, or 6. The distribution type was either student-t or normal, this was due to limitations set by the original code base [43]. It was observed that a forecasting length of 24, 200, 225, and 300 were not able to predict the sudden change in trace data from high to low. Thus, they will not be included in the analysis. It was also observed that the models that were able to capture that sudden change had a scaled MSE value less than 0.09. The lowest trace metrics from the model with the lowest mean metrics (over all of the traces) is included in Table 6 and its predictions and mean metrics per trace are displayed in Figure 40.

Table 6. Power draw Probabilistic TNN prediction results for the unscaled trace data. The model's hyperparameters are four encoder and decoder layers, with a normal distribution, a *d_model* of 64, and 40 traces. The forecasting length was 65. The traces that produced the lowest MSE and MAE and their values are listed.

Trace Number	MSE	MAE
28	**619.461**	16.513
4	677.708	**15.279**

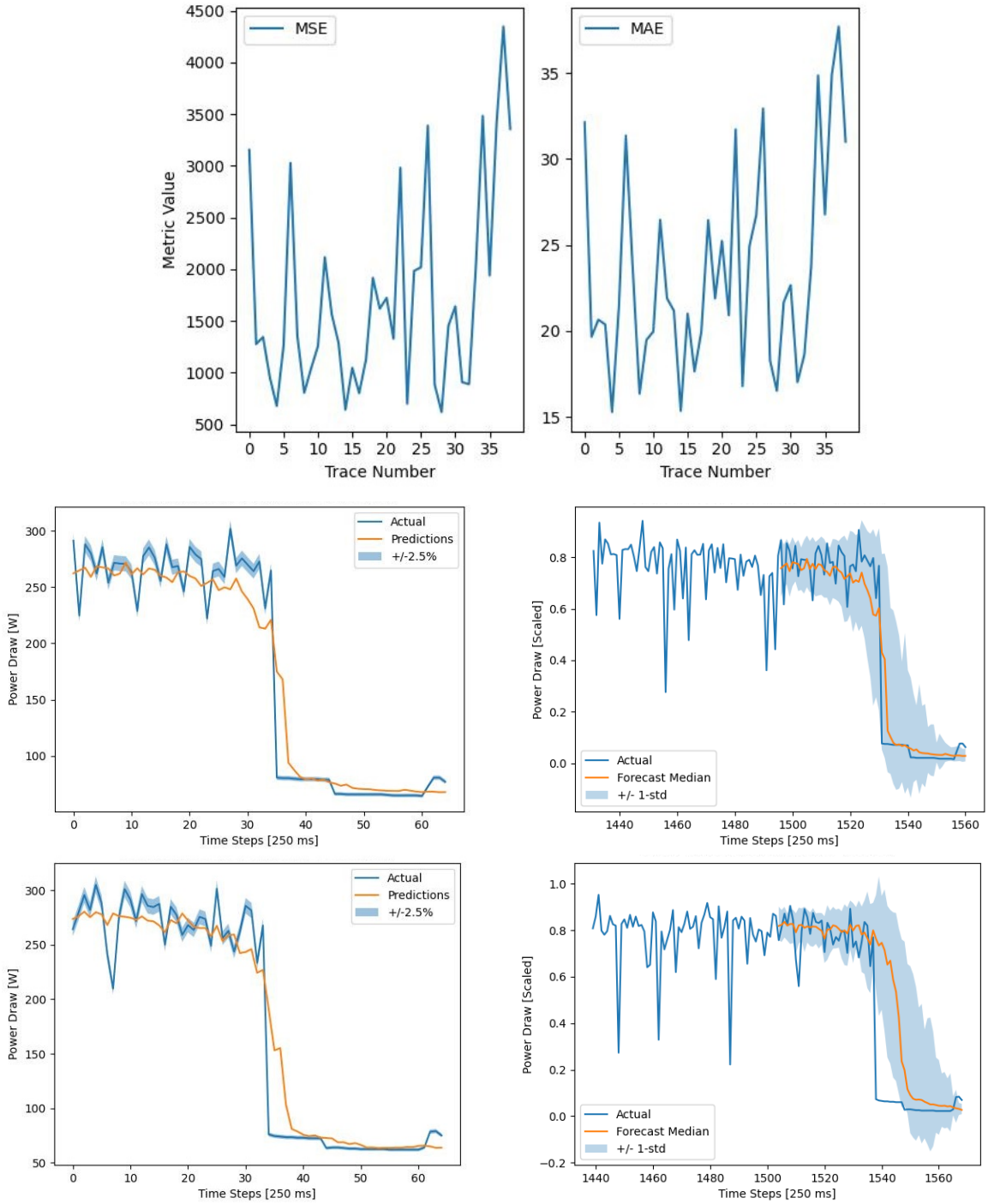

Figure 40. Power draw Probabilistic TNN prediction results and metrics per trace. Top: MAE and MSE metrics per trace for the model hyperparameters that produced the lowest MAE and MSE metrics. Middle: prediction results for Trace 28, which produced the lowest MSE. Unscaled on the left and scaled on the right (with uncertainty +/- 1 standard deviation). Bottom: prediction results for Trace 4, which produced the lowest MAE. Unscaled on the left and scaled on the right (with uncertainty +/- 1 standard deviation).

4.3.3 Annotated TNN GPU Utilization Results

For each set of hyperparameters, the model that resulted in the lowest mean MSE, lowest mean MAE, lowest minimum MSE, and lowest minimum MAE values was listed in Table 7. Forecasting lengths of 24 were not able to capture the sudden changes in trace data. The best model utilized a forecast amount of 160 and smoothing of 0.025, 40 traces, and 4 layers. Figure 41 displays the batch that resulted in the lowest MAE and MSE metrics.

Table 7. Prediction evaluation metric summary on GPU utilization for the Annotated TNN model. The model resulting in the most optimal metric value is included, along with some of the model's hyperparameters.

Metric Name	Forecasting Length, Number of Layers, Smoothing	Metric Value
Mean MSE	Length = 48, Layers = 4, Smoothing = 0.025	758.91
Mean MAE	Length = 48, Layers = 4, Smoothing = 0.025	22.88

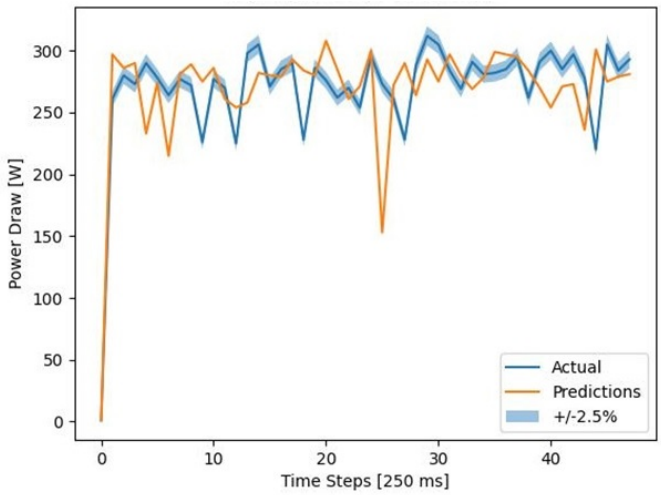

Figure 41. This is the batch from the test set that resulted in the lowest MSE, and MAE.

One of the major downsides to this type of model is that the metric results for each batch were vastly different.

4.3.4 Annotated TNN Power Draw Results

As explained in Section 3.4.2, the inputs to the model are not scaled and are in integer form. This model was constructed to distribute the work among three parallel GPUs. An all-reduce averaging allows the collection of the prediction metrics after the model is fully trained. For each of the batches in the test data set, distributed among the parallel GPUs, the batch that resulted in the lowest MAE and MSE scores were plotted.

For each set of hyperparameters, the model that resulted in the lowest mean MSE and lowest mean MAE are listed in Table 8. The hyperparameter search for this TNN type was limited to just varying the number of layers and the amount of smoothing for the Label Smoother, either 0.025 or 0.1.

Table 8. Power draw Annotated TNN prediction evaluation metric results. Prediction evaluation metric summary on power draw for the Annotated TNN. The model resulting in the most optimal metric value is included, along with some of the model's hyperparameters.

Metric Name	Forecasting Length, Number of Layers	Metric Value
Mean MSE	Len. = 48, Layers = 4, 0.025	1103.90
Mean MAE	Len. = 48, Layers = 4, 0.025	24.06

Figure 42 displays the actual and predictions for the batch that resulted in the lowest MSE and MAE.

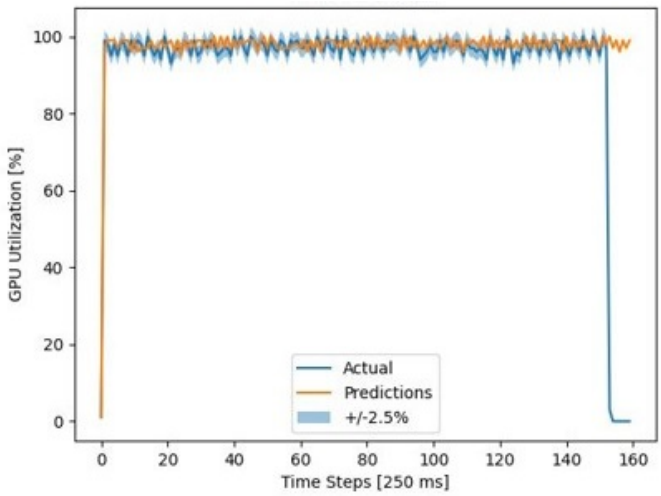

Figure 42. Power draw Annotated TNN prediction results. This is the batch from the test set that resulted in the lowest MSE and MAE metrics.

For this type of TNN, metrics for the same type of model, with different forecasting horizon lengths varied greatly. Even within a model, batch results varied greatly as well. This type of TNN performed even worse than the results for the GPU utilization prediction.

4.4 FFORMA RESULTS

Experimental results were collected for both the GPU utilization and power draw trace as univariate time series traces. To make it easier to compare the models, the prediction evaluation metrics were MAE, MSE, and RMSE. The prediction evaluation metrics listed in the summary tables are the average of the metrics over each of the traces in the train data set. The trace data was not transformed. The weights for the meta-model on each of the time series prediction models in the pool are collected.

Analysis was done on the hyperparameter set that resulted in the lowest mean MSE, MAE, and RMSE. Then on the trace resulting in the lowest mean MSE, MAE, and RMSE.

For both the GPU utilization and power draw trace prediction models, the forecasting horizon was either 48, 65, or 160. The traces were either input as a seasonal or not seasonal time series. For seasonal time series inputs, the frequency was either 3, 4, or 8.

4.4.1 FFORMA: GPU Utilization Results

A singular model produced the most optimal mean SMAPE, MSE, MAE, and RMSE values. The model provided FFORMA the input with a forecasting length of 160 and a seasonal frequency of 3. This model's mean metrics per trace are displayed in Figure 43 and are displayed in Table 9. Figure 44 displays the prediction results. On the bottom of the figure, are the two trace results that resulted in the lowest SMAPE and MAE scores on the bottom left and the lowest MSE and RMSE on the bottom right.

Table 9. Prediction evaluation metric summary on GPU utilization for FFORMA. The model resulting in the most optimal metric value is included, with the model's hyperparameters.

Metric Name	Forecasting Length, Seasonal Frequency	Metric Value
Mean SMAPE	Len. = 160, Seasonal Freq. = 3	0.488
Mean MAE	Len. = 160, Seasonal Freq. = 3	27.853
Mean MSE	Len. = 160, Seasonal Freq. = 3	2294.687
Mean RMSE	Len. = 160, Seasonal Freq. = 3	47.074

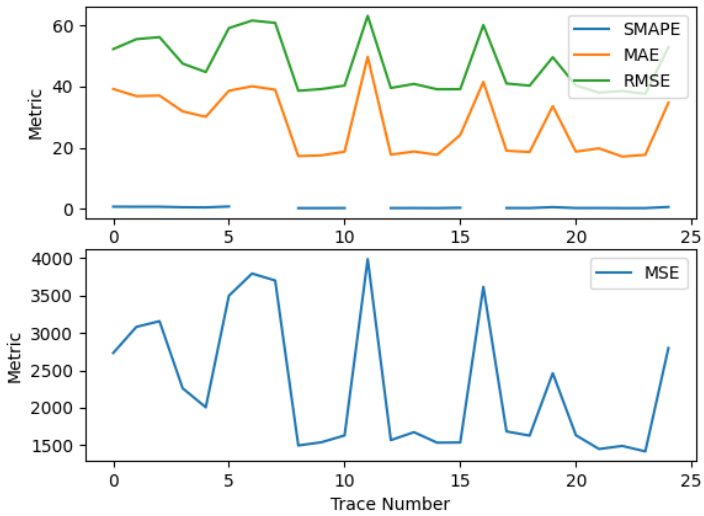

Figure 43. GPU utilization FFORMA prediction results. The mean result metrics per trace for the model that produced the lowest mean SMAPE, MSE, MAE, and RMSE values.

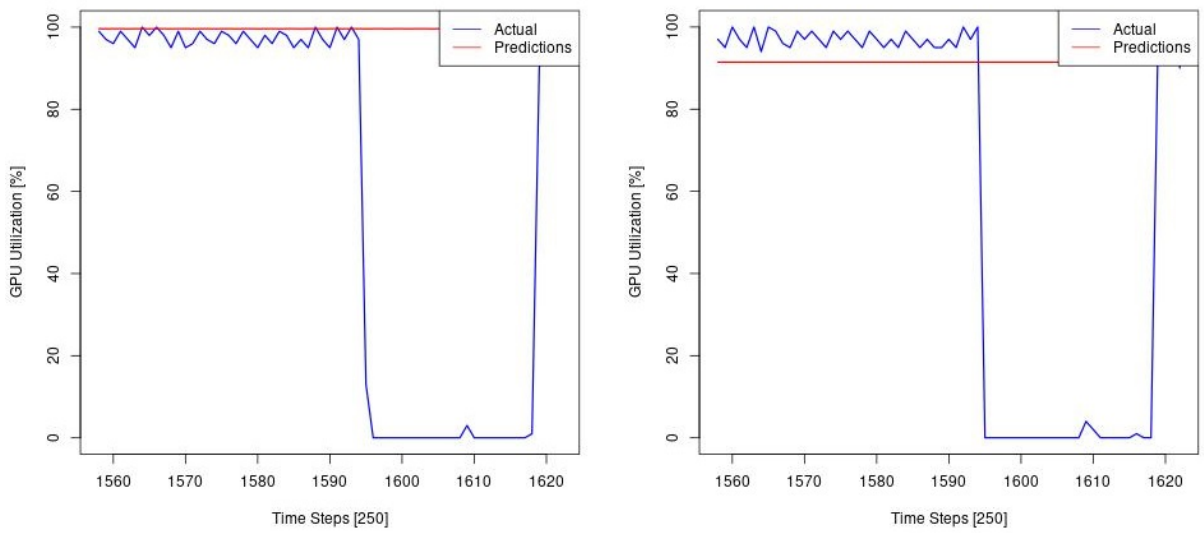

Figure 44. GPU utilization FFORMA prediction results. Left: model's trace that produced the lowest MSE and RMSE values. Right: model's trace that produced the lowest SMAPE and MAE.

4.4.2 FFORMA: Power Draw Results

The prediction model producing the lowest mean prediction evaluation metrics SMAPE, MAE, MSE, and RMSE input the trace series as non-seasonal time series. The prediction model resulting in the lowest mean SMAPE value produced the same results (for the SMAPE value) with the power draw trace as a seasonal time series with a frequency of four. The prediction length for the models was 24. The predictions for the model that produced the lowest SMAPE, MAE, MSE, and RMSE metric values are displayed in Figure 45 and included in Table 10.

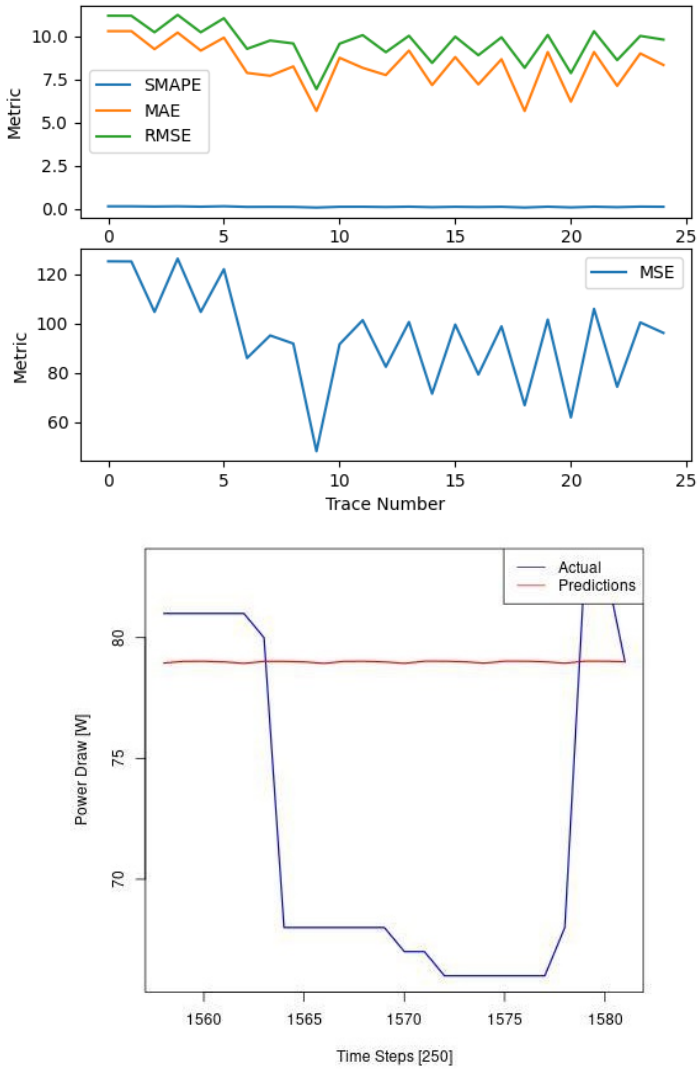

Figure 45. Power draw evaluation prediction results. Top: The mean result metrics per trace for the model that produced the lowest mean SMAPE, MSE, MAE, and RMSE values. On the bottom, are the actual and prediction values for the model that produced the lowest average prediction evaluation metrics MSE, MAE, and RMSE. The specific test set shown belongs to the data trace that resulted in the lowest metrics.

It is interesting that changing the input time series from seasonal to not seasonal produced only a slight difference in evaluation metrics. This suggests that the weights assigned to the seasonal time series prediction models used in the pool of models to consider were already low. This was verified in Figure 46.

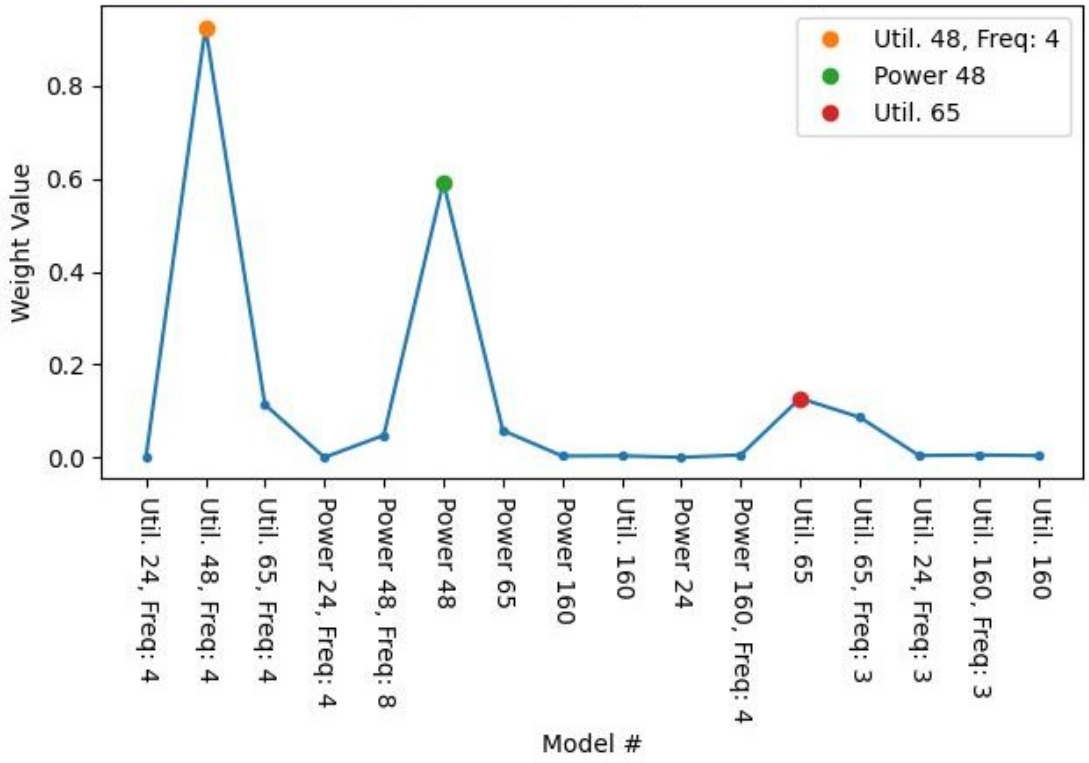

Figure 46. This figure shows the weights for the meta-model for each of the models created using a different set of hyperparameters. The top three maximum values are shown in the table.

Table 10. FFORMA power draw model producing the lowest metric is listed along with its set of hyperparameters and value.

Metric Name	Forecasting Length, Seasonal Frequency	Metric Value
Mean SMAPE	Len. 24, Not Seasonal	0.118
Mean MAE	Len. 24, Not Seasonal	8.211
Mean MSE	Len. 24, Not Seasonal	92.628
Mean RMSE	Len. 24, Not Seasonal	9.561

4.5 RESULT DISCUSSIONS

4.5.1 Discussion: GPU Utilization Models

Multivariate models generally match or outperform univariate models when there is a pattern/correlation between the variables [57]. The GPU utilization test prediction evaluation metrics for the Univariate and Multivariate RNN models (see Table 11, lines 2 and 3) produced a lower MAE and lower MSE respectively. Prediction results in Figures 25 and 33 verified this observation. MSE heavily penalized the 20+ GPU utilization difference between prediction and actual values near the point of sudden change. According to MAE, the multivariate model did not improve with the additional power draw trace. However, according to MSE, the additional power draw trace improved predictions, implying that there was a pattern between the traces that improved accuracy. Both the univariate and multivariate models showed improvements over the baseline model. The Probabilistic TNN produced prediction evaluation metrics lower than those produced by the RNN prediction models but still produced metrics lower than the baseline. The FFORMA GPU utilization and Annotated TNN models did not produce very accurate predictions. FFORMA provides more insight into the trace data. The FFORMA results are not in Table 11 since the results above did not remove poorly captured sudden changes in trace data.

Table 11. GPU utilization prediction model summary. Each model type's training speed (training epochs per second), and the resulting MSE and MAE. No epochs are represented by a $\star$. No timing speed (since the model is lagging with respect to MSE and MAE) is represented by a †.

GPU Utilization Model	Epoch/Sec	MSE	MAE
Baseline	$\star$	2147.96	24.42
Univariate RNN (Table 1.12)	13.41	177.21	**3.72**
Multivariate RNN (Table 3.12)	19.99	**139.29**	3.74
Probabilistic TNN (Table 6)	24.49	1358.29	16.89
Annotated TNN (Table 7)	†	758.91	22.88

4.5.2 Discussion: Power Draw Models

The univariate RNN model produced a lower MSE value, but a larger MAE value than the multivariate RNN model, as shown in Table 12 lines 2 and 3. In other words, the additional GPU utilization trace data increased the MSE, but decreased the MAE. The Probabilistic TNN model outperformed both the univariate and multivariate RNN models in terms of both MSE and MAE. The TNN model was faster during training than both RNN models. The multivariate was slower than the univariate.

The Annotated TNN model produced a lower MSE score than the baseline, but a larger MAE value. This is likely because this model was optimized for translation tasks. The FFORMA model produced very large metrics compared with the baseline model. However, this model provides good insight into each trace's time series characteristics. The FFORMA results are not in Table 12 since the results above did not remove poorly captured sudden changes in trace data.

Table 12. Power draw prediction model summary. Each model type's training speed (training epochs per second) and resulting MSE and MAE metrics. No epochs are represented by a $\star$. No timing speed (since the model is lagging with respect to MSE and MAE) is represented by a †.

Power Draw Model	Epoch/Sec	MSE	MAE
Baseline	$\star$	1263.52	21.03
Univariate RNN (Table 2.12)	9.38	1099.70	17.87
Multivariate RNN (Table 4.12)	9.80	1173.05	17.25
Probabilistic TNN (Table 5.2)	8.72	**677.71**	**15.28**
Annotated TNN (Table 8)	†	1103.90	24.06

CHAPTER 5

CONCLUSIONS AND FUTURE WORK

Time series analysis revealed that neither original GPU utilization nor power draw traces were stationary. The GPU utilization trace required a differencing and the power draw required a detrending to produce a stationary time series. Experimental results on the statistical-based models $AR(p)$, $MA(q)$, $ARMA(p,q)$, $ARIMA(p,d,q)$, $SARIMA(p,d,q)(P,D,Q)[m]$, and models selected from auto-ARIMA, revealed that the auto-regressive models with lags equating to the last significant lag from the partial auto-correlation graphs of the stationary series (for both GPU utilization and power draw) outperformed the other statistical models considered in terms of the prediction evaluation metrics of MSE, MAE, and RMSE. The moving average models did not perform well in terms of the evaluation metrics. For both the stationary GPU utilization and power draw traces, the selected seasonal models outperformed the selected non-seasonal models, but neither performed better than the $AR(p)$ models, in terms of the prediction evaluation metrics MSE, MAE, and RMSE.

Experimental results for the multivariate versions of the models demonstrated that the additional trace data only assisted the prediction accuracy for test MSE, RMSE, and "Within 2.5%" metrics for predicting the GPU utilization trace. And test "Within 2.5%" and "Within 1.0%" metrics for predicting the power draw trace. The multivariate forms of the models outperformed their univariate counterparts, and the statistical-based models for the MSE and RMSE metrics.

The Probabilistic TNN produced the lowest MSE and MAE metrics for the power draw trace. However, it did not preform well for the GPU utilization trace. This was because the power draw Probabilistic TNN was able to capture the sudden changes, but struggled to do so for the GPU utilization trace. The Annotated Transformer Neural Network was unable to properly map the inputs to the outputs for the power draw trace. This was not surprising, considering it was optimized for translation problems. It did however perform better on the GPU utilization data.

RNN model results provided the most improvement compared to the statistical-based

model for predictions of the GPU utilization trace, as shown in Table 11. The lowest MSE was produced by the multivariate RNN, and the lowest MAE was produced by the univariate RNN. The Probabilistic TNN for the power draw model produced both the lowest MAE and MSE metrics, especially compared to the baseline statistical-based models, as demonstrated in Table 12.

5.0.1 Future Work

Future work includes producing experimental results on online machine-learning algorithms to test the hypothesis that these types of machine-learning algorithms produce more accurate predictions. Future work also includes utilizing the predictions to set a maximum power, to hopefully produce power-efficiency.

REFERENCES

[1] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *2016 IEEE Conference on Computer Vision and Pattern Recognition*, *27-30 June 2016, Las Vegas, NV*, pp. 2818–26, [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7780677. [Accessed Apr. 29, 2024].

[2] S. C. Hoi, D. Sahoo, J. Lu, and P. Zhao, "Online learning: A comprehensive survey," *Neurocomputing*, vol. 459, pp. 249–289, Oct. 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231221006706. [Accessed Apr. 29, 2024].

[3] D. Zhao, S. Samsi, J. McDonald, B. Li, D. Bestor, M. Jones, D. Tiwari, and V. Gadepally, "Sustainable supercomputing for AI: GPU power capping at HPC scale," in *Proceedings of the 2023 ACM Symposium on Cloud Computing*, *Oct. 2023, New York, NY*, pp. 588–596, [Online]. Available: https://doi.org/10.1145/3620678.3624793. [Accessed Apr. 29, 2024].

[4] J. McDonald, B. Li, N. Frey, D. Tiwari, V. Gadepally, and S. Samsi, "Great power, great responsibility: Recommendations for reducing energy for training language models," in *Findings of the Association for Computational Linguistics: NAACL 2022, 10-15 July 2022*, *Seattle, WA*, M. Carpuat, M. de Marneffe, and I. V. Meza Ruiz Eds., pp. 1962–70, [Online]. Available: https://aclanthology.org/2022.findings-naacl.151. [Accessed Apr. 29, 2024].

[5] R. Drechsler, C. A. Metz, and C. Plump, "Energy-efficient CNN inferencing on GPUs with dynamic frequency scaling," *International Conference on Innovations in Data Analytics*, 2023. [Online]. Available: https://agra.informatik.uni-bremen.de/doc/konf/ICIDA2023_CM.pdf. [Accessed Apr. 29, 2024].

[6] D. Kang, Y. Ha, L. Peng, and C. Youn, "Cooperative distributed GPU power capping for deep learning clusters," *IEEE Transactions on Industrial Electronics*, vol. 69, no. 7, pp. 7244–54, July 2022. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9484692. [Accessed Apr. 29, 2024].

[7] A. K. Shaikh, A. Nazir, I. Khan, and A. S. Shah, "Short term energy consumption forecasting using neural basis expansion analysis for interpretable time series," *Scientific Reports*, vol. 12, no. 22562, December 2022. [Online]. Available: https://europepmc.org/articles/PMC9799709. [Accessed Apr. 29, 2024].

[8] H. S. Oliveira and H. P. Oliveira, "Transformers for energy forecast," *Sensors*, vol. 23, no. 15, 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/15/6840. [Accessed Apr. 29, 2024].

[9] R. A. Bridges, N. Imam, and T. M. Mintz, "Understanding GPU power: A survey of profiling, modeling, and simulation methods," *ACM Computing Surveys*, vol. 49, no. 3, pp. 1–27, Sept. 2016. [Online]. Available: https://doi.org/10.1145/2962131. [Accessed Apr. 29, 2024].

[10] N. Wu, B. Green, X. Ben, and S. O'Banion, "Deep transformer models for time series forecasting: The influenza prevalence case," *arXiv preprint*, 2020. [Online]. Available: https://arxiv.org/abs/2001.08317. [Accessed Apr. 29, 2024].

[11] A. Verma, "Performance engineering of transformers on CPU vs GPU," Master's thesis, Friedrich-Alexander-Universität Erlangen-Nürnberg, Aug. 2023. [Online]. Available: https://www10.cs.fau.de/publications/theses/2023/Master_VermaAmritanshu.pdf. [Accessed Apr. 29, 2024].

[12] B. Dutta, V. Adhinarayanan, and W. Feng, "GPU power prediction via ensemble machine learning for DVFS space exploration," in *Proceedings of the 15th ACM International*

Conference on Computing Frontiers, ser. CF '18, New York, NY: Association for Computing Machinery. 2018. ISBN 9781450357616 pp. 240–243, [Online]. Available: https://doi.org/10.1145/3203217.3203273. [Accessed Apr. 29, 2024].

[13] G. Yeung, D. Borowiec, A. Friday, R. Harper, and P. Garraghan, "Towards GPU utilization prediction for cloud deep learning," in *Proceedings of the 12th USENIX Conference on Hot Topics in Cloud Computing*, ser. HotCloud'20, USA: USENIX Association. July 2020. [Online]. Available: https://dl.acm.org/doi/pdf/10.5555/3485849.3485855. [Accessed Apr. 29, 2024].

[14] V. Sundriyal and M. Sosonkina, "Runtime energy savings based on machine learning models for multicore applications," *Journal of Computer and Communications*, vol. 10, no. 6, pp. 63–80, 2022. [Online]. Available: https://www.scirp.org/journal/paperinformation.aspx?paperid=118212. [Accessed Apr. 29, 2024].

[15] M. Sosonkina, V. Sundriyal, and J. Galvez Vallejo, "Runtime power allocation based on multi-GPU utilization in GAMESS," *Journal of Computer and Communications*, vol. 10, pp. 66–80, 2022. [Online]. Available: https://scirp.org/journal/paperinformation.aspx?paperid=119960. [Accessed Apr. 29, 2024].

[16] S. Mittal and J. Vetter, "A survey of methods for analyzing and improving GPU energy efficiency," *ACM Computer Survey*, vol. 47, no. 2, pp. 1–23, Aug. 2014. [Online]. Available: https://doi.org/10.1145/2636342. [Accessed Apr. 29, 2024].

[17] "Green-500 List," Top500.org, Nov. 2022. [Online]. Available: https://www.top500.org/lists/green500/2022/11/. [Accessed Apr. 29, 2024].

[18] "Frontier: Direction of discovery," Oak Ridge National Laboratory, 2022. [Online]. Available: https://www.olcf.ornl.gov/frontier/#4. [Accessed Apr. 29, 2024].

[19] "nvidia-smi - nvidia system management interface program," NVIDIA Corporation, 2016. [Online]. Available: https://developer.download.nvidia.com/compute/DCGM/docs/nvidia-smi-367.38.pdf. [Accessed Apr. 29, 2024].

[20] E. Can, R. Arora, and P. Chitale, "Profiling and optimizing deep neural networks with DLProf and PyProf," Sept. 2020. [Online]. Available: https://developer.nvidia.com/blog/profiling-and-optimizing-deep-neural-networks-with-dlprof-and-pyprof/. [Accessed Apr. 29, 2024].

[21] M. Steurer, R. J. Hill, and N. Pfeifer, "Metrics for evaluating the performance of machine learning based automated valuation models," *Journal of Property Research*, vol. 38, no. 2, pp. 99–129, Apr. 2021. [Online]. Available: https://doi.org/10.1080/09599916.2020.1858937. [Accessed Apr. 29, 2024].

[22] "Lambda TensorFlow benchmark," GitHub, 2021. [Online]. Available: https://github.com/lambdal/lambda-tensorflow-benchmark. [Accessed Apr. 29, 2024].

[23] "XLA: Optimizing compiler for machine learning," 2022. [Online]. Available: https://www.tensorflow.org/xla. [Accessed Apr. 29, 2023].

[24] "Nvidia collective communications library (NCCL)," NVIDIA Developer, 2023. [Online]. Available: https://developer.nvidia.com/nccl. [Accessed Apr. 29, 2024].

[25] "Deep learning GPU benchmarks," Lambda Labs, 2024. [Online]. Available: https://lambdalabs.com/gpu-benchmarks. [Accessed Apr. 29, 2024].

[26] scikit learn, "sklearn.preprocessing.minmaxscaler," 2024. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html. [Accessed Apr. 29, 2024].

[27] J. Brownlee, “How to use data scaling improve deep learning model stability and performance,” August 25, 2020. [Online]. Available: https://machinelearningmastery.com/how-to-improve-neural-network-stability-and-modeling-performance-with-data-scaling/. [Accessed Apr. 29, 2024].

[28] M. Sosonkina, “High performance computing and simulation,” class notes for High Performance Computing and Simulation, Dept. of Modeling and Simulation, Old Dominion University, Norfolk, VA, Spring 2023.

[29] M. Peixeiro, *Time Series Forecasting in Python*, Manning. 2022. ISBN 9781617299889 [Online]. Available: https://www.manning.com/books/time-series-forecasting-in-python-book.

[30] statsmodels, “Stationarity and detrending (ADF/KPSS),” statsmodels 0.15.0, March 29, 2024. [Online]. Available: https://www.statsmodels.org/dev/examples/notebooks/generated/stationarity_detrending_adf_kpss.html. [Accessed Apr. 29, 2024].

[31] H. Yang, “Advanced analysis for modeling & simulation,” PowerPoint Slides, Dept. of Modeling and Simulation, Old Dominion University, Norfolk, VA, Fall 2023.

[32] R. J. Hyndman and G. Athanasopoulos, *Forcasting: Principles and Practice, 2nd ed.*, OTexts: Melbourne, Australia: Texts.com/fpp2. 2018. [Online]. Available: https://otexts.com/fpp2/. [Accessed Apr. 29, 2024].

[33] T. G. Smith, “pmdarima.arima.auto_arima,” 2023. [Online]. Available: https://alkaline-ml.com/pmdarima/modules/generated/pmdarima.arima.auto_arima.html. [Accessed Apr. 29, 2024].

[34] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, Boston, MA: O’Reilly Media, Inc. 2019. ISBN 978-1-492-03264-9

[35] S. Sharma, S. Sharma, and A. Athaiya, “Activation functions in neural networks,” *International Journal of Engineering Applied Sciences and Technology*, vol. 4, no. 12, pp. 310–316, 2020. [Online]. Available: https://www.ijeast.com/papers/310-316,Tesma412,IJEAST.pdf. [Accessed Apr. 29, 2024].

[36] B. Momin and G. Chavan, “Univariate time series models for forecasting stationary and non-stationary data: A brief review,” in *Information and Communication Technology for Intelligent Systems - Volume 2*, ser. ICTIS, 2017, Springer International Publishing. Aug. 2018. S. Satapathy and A. Joshi Eds., vol. 2. ISBN 978-3-319-63644-3 pp. 219–26, [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-63645-0_24. [Accessed Apr. 29, 2024].

[37] M. Saeed, “An introduction to recurrent neural networks and the math that powers them,” Machine Learning Mastery, Jan. 2023. [Online]. Available: https://machinelearningmastery.com/an-introduction-to-recurrent-neural-networks-and-the-math-that-powers-them/. [Accessed Apr. 29, 2024].

[38] M. Saeed, “Understanding simple recurrent neural networks in keras,” Machine Learning Mastery, Jan. 2023. [Online]. Available: https: //machinelearningmastery.com/understanding-simple-recurrent-neural-networks-in-keras/. [Accessed Apr. 29, 2023].

[39] F. Chollet, *Deep Learning with Python*, Shelter Island, NY: Manning. 2021. ISBN 978-1-61729-686-4

[40] B. Lindemann, T. Müller, H. Vietz, N. Jazdi, and M. Weyrich, “A survey on long short-term memory networks for time series prediction,” *Procedia CIRP*, vol. 99, pp. 650–5, 15-17 July 2020. 14th CIRP Conference on Intelligent Computation in Manufacturing Engineering. [Online]. Available:

https://www.sciencedirect.com/science/article/pii/S2212827121003796. [Accessed Apr. 29, 2024].

[41] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, ser. NeurIPS 30, 2017, [Online]. Available: https://arxiv.org/pdf/1706.03762. [Accessed Apr. 29, 2024].

[42] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient transformers: A survey," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–28, Dec. 2022. [Online]. Available: https://doi.org/10.1145/3530811. [Accessed Apr. 29, 2024].

[43] N. Rogge and K. Rasul, "Probabilistic time series forecasting with transformers," Hugging Face, Dec. 2022. [Online]. Available: https://huggingface.co/blog/time-series-transformers. [Accessed Apr. 29, 2024].

[44] PyTorch, "Welcome to pytorch tutorials," PyTorch, 2017. [Online]. Available: https://brsoff.github.io/tutorials/index.html. [Accessed Apr. 29, 2024].

[45] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan, "Accelerate: Training and inference at scale made simple, efficient and adaptable." 2022. [Online]. Available: https://github.com/huggingface/accelerate. [Accessed Apr. 29, 2024].

[46] A. Alexandrov, K. Benidis, M. Bohlke-Schneider, V. Flunkert, J. Gasthaus, T. Januschowski, D. C. Maddix, S. Rangapuram, D. Salinas, J. Schulz, L. Stella, A. C. Türkmen, and Y. Wang, "GluonTS: Probabilistic and Neural Time Series Modeling in Python," *Journal of Machine Learning Research*, vol. 21, no. 116, pp. 1–6, Apr. 2020. [Online]. Available: http://jmlr.org/papers/v21/19-820.html. [Accessed Apr. 29, 2024].

[47] "Gluonts documentation: gluonts.transform.sampler module," GluonTS, 2022. [Online]. Available: https://ts.gluon.ai/stable/api/gluonts/gluonts.transform.sampler.html. [Accessed Apr. 29, 2024].

[48] K. G. A. Ludvigsen, R. Holgarsson, P. H., and M. S. Mathew, “Influenza transformer,” GitHub Repository, 2022. [Online]. Available: https://github.com/KasperGroesLudvigsen/influenza_transformer. [Accessed Apr. 29, 2024].

[49] A. Rush, “The annotated transformer,” in *Proceedings of Workshop for NLP Open Source Software (NLP-OSS)*, Melbourne, Australia: Association for Computational Linguistics. Jul. 2018. E. L. Park, M. Hagiwara, D. Milajevs, and L. Tan Eds., pp. 52–60, [Online]. Available: https://aclanthology.org/W18-2509. [Accessed Apr. 29, 2024].

[50] A. Huang, S. Subramanian, J. Sum, K. Almubarak, and S. Biderman, “The annotated transformer,” 2022. [Online]. Available: https://nlp.seas.harvard.edu/annotated-transformer/. [Accessed Apr. 29, 2024].

[51] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “The M4 competition: 100,000 time series and 61 forecasting methods,” *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, 2020. M4 Competition. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169207019301128. [Accessed Apr. 29, 2024].

[52] P. Montero-Manso, G. Athanasopoulos, R. J. Hyndman, and T. S. Talagala, “FFORMA: Feature-based forecast model averaging,” *International Journal of Forecasting*, vol. 36, no. 1, pp. 86–92, 2020. M4 Competition. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169207019300895. [Accessed Apr. 29, 2024].

[53] P. Montero-Manso, “Fforma: Feature-based forecast model averaging,” 2020. [Online]. Available: https://github.com/pmontman/fforma. [Accessed Apr. 29, 2024].

[54] P. Cawood and T. Van Zyl, “Evaluating state-of-the-art, forecasting ensembles and meta-learning strategies for model fusion,” *Forecasting*, vol. 4, no. 3, pp. 732–51, 2022. [Online]. Available: https://www.mdpi.com/2571-9394/4/3/40. [Accessed Apr. 29, 2024].

[55] R. Hyndman, Y. Kang, P. Montero-Manso, M. O’Hara-Wild, T. Talagala, E. Wang, and Y. Yang, *tsfeatures: Time Series Feature Extraction*, 2023. [Online]. Available: https://pkg.robjhyndman.com/tsfeatures/. [Accessed Apr. 29, 2024].

[56] A. Kapoor, A. Gulli, and S. Pal, *Deep Learning with TensorFlow and Keras, 3rd Edition*, Packt. 2021. ISBN 9781803232911

[57] W. M. J. Murphy and K. Chen, “Univariate vs multivariate time series forecasting with transformers,” in *The Eleventh International Conference on Learning Representations (ICLR 2023)*, Feb. 2023, [Online]. Available: https://openreview.net/forum?id=GpW327gxLTF. [Accessed Apr. 29, 2024].

VITA

Dorothy Xiaoshuang Parry
Department of Modeling and Simulation Engineering
Old Dominion University
Norfolk, VA 23529

EDUCATION

MS in Modeling and Simulation Engineering, Old Dominion University, Norfolk, VA. May 2024

BS in Engineering Physics and Mathematics, Randolph-Macon College, Ashland, VA. June 2018

PROFESSIONAL EXPERIENCE

Scientist at Naval Surface Warfare Center Dahlgren Division, worked as an analyst and agile Java developer on a cross-functional team for a complex, multi-layered optimizer.

Research Assistant at Old Dominion University, researched predictive time series for the purpose of High-Power Computing (HPC) parallel GPU performance analysis.

Research Assistant at Old Dominion University, worked on a group research project that explored autonomous navigation of a maritime vessel through machine learning-based buoy classification of LiDAR point clouds.

Workshop Teacher's Assistant at Old Dominion University, developed and redesigned the module titled "Parallel and High-Performance Computing," a lesson contained within ODU's DeapSECURE cyberinfrastructure training.